UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

LAB 4 · Framework Multiplataforma

MenúRadar

Prototipo móvil en Flutter para encontrar menús económicos y disponibles cerca del campus



Integrantes

Jorge Luis Escobedo Ocaña
Jose Alonzo Gordillo Mendoza

Docente

Norman Patrick Harvey Arce

Arequipa, Octubre de 2026

Índice de contenido

1. Descripción general del programa....4
1.1 Contexto y problema....4
1.2 Objetivo y alcance....4
1.3 Funcionalidades....5
1.4 Herramientas y versiones....5
2. Diseño de interfaces (funcionalidades y mockup)....6
2.1 Menús....7
2.2 Detalle del menú....8
2.3 Radar....9
2.4 Comunidad....10
2.5 Publicar menú....12
3. Descripción del proceso de desarrollo....13
3.1 Preparación del entorno....13
3.2 Estructura del proyecto....13
3.3 Cómo Flutter organiza el layout....14
3.4 Construcción de cada pantalla....14
3.5 Tipografía y tema....16
3.6 Ilustraciones con CustomPainter....17
3.7 Pruebas y control de versiones....18
4. Apreciación de la herramienta....19
4.1 Características a favor....19
4.2 Características en contra....19
Anexos: código de la aplicación....20
Referencias....59

Índice de imágenes

Figura 1. Árbol de problemas de MenúRadar (causas, problema central y efectos).....4
Figura 2. Mockup general: Menús, Detalle, Radar, Comunidad y Publicar menú.....6
Figura 3. Pantalla Menús.....7
Figura 4. Menús al desplazar la lista.....7
Figura 5. Tarjeta de un menú agotado.....7
Figura 6. Detalle con la cabecera expandida.....8
Figura 7. Detalle con la cabecera colapsándose.....8
Figura 8. Radar sin selección.....9
Figura 9. Radar con un local seleccionado.....9
Figura 10. Comunidad con los avisos de ejemplo.....10
Figura 11. Comunidad tras publicar un aviso propio.....10
Figura 12. Hoja Nuevo aviso (primera vez, pide apodo).....11
Figura 13. Hoja Nuevo aviso con los datos completos.....11
Figura 14. Formulario al abrir.....12
Figura 15. Vista previa con nombre y precio.....12
Figura 16. Cocina «Otro» seleccionada.....12
Figura 17. Las cuatro ilustraciones: plato criollo, bowl vegetariano, bowl de chifa y cloche genérica.
.....17

Índice de tablas

Tabla 1. Funcionalidades del prototipo, la causa del problema que atacan y la pantalla que las contiene.....5
Tabla 2. Herramientas utilizadas y versiones comprobadas en el equipo de desarrollo.....5
Tabla 3. Opciones para identificar a quien publica un aviso en Comunidad.....10
Tabla 4. Organización de la carpeta `lib/` y responsabilidad de cada archivo.....13
Tabla 5. Widgets de organización y layout estudiados, su función y dónde se usaron.....14
Tabla 6. Tipografías del proyecto.....16
Tabla 7. Ilustración de cada cocina y las formas con que se dibuja.....17
Tabla 8. Verificaciones realizadas sobre el proyecto.....18

1. Descripción general del programa

1.1 Contexto y problema

MenúRadar es una aplicación móvil pensada para estudiantes universitarios. Su propósito es que puedan **comparar menús económicos y saber cuáles siguen disponibles** antes de salir del campus durante su receso. Parte de un trabajo previo del equipo desarrollado como aplicación web (PWA con React); en este laboratorio se rediseña la experiencia con **Flutter**, un framework multiplataforma, usando únicamente los componentes propios de la plataforma.

El diseño de las interfaces se deriva del siguiente árbol de problemas. La causa que más pesa es que la información de platos, precios y disponibilidad vive en pizarras físicas y cambia sin aviso, por lo que el estudiante solo se entera de que un plato se agotó cuando ya caminó hasta el local.

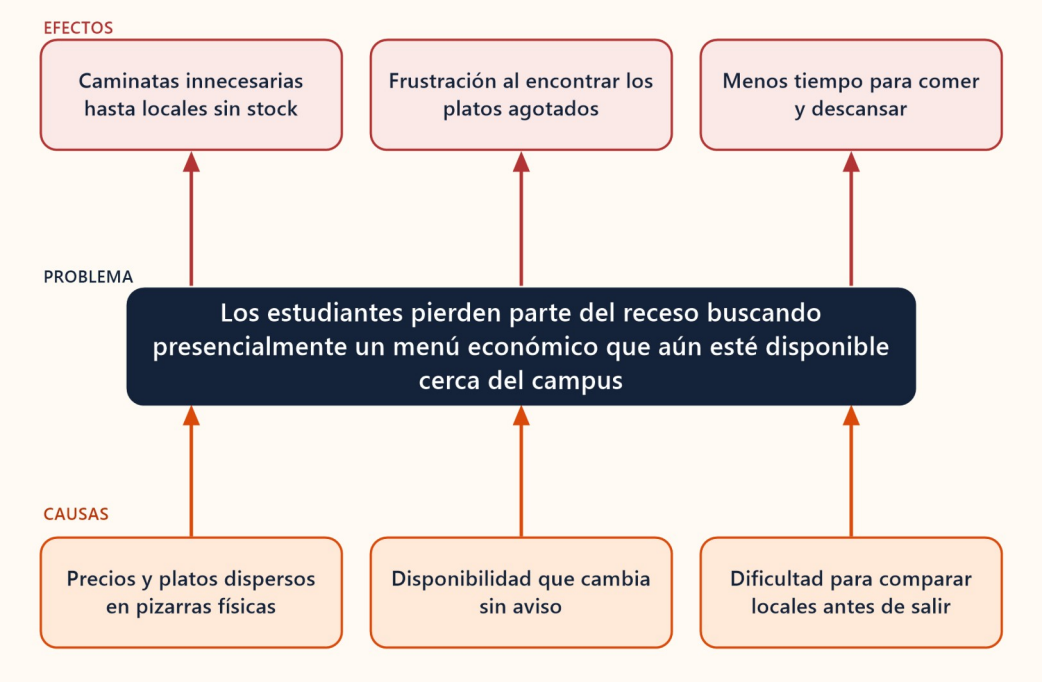


Figura 1. Árbol de problemas de MenúRadar (causas, problema central y efectos).

1.2 Objetivo y alcance

El objetivo es construir un prototipo funcional cuyas pantallas respondan a cada causa del árbol de problemas y que sirva para estudiar cómo Flutter organiza una interfaz y maneja el layout.

- **Incluido:** búsqueda y filtros de menús, detalle de cada menú, radar de locales, avisos de la comunidad, publicación de un menú y reporte de disponibilidad.
- **No incluido:** base de datos, cuentas de usuario, GPS, cámara ni sincronización entre dispositivos. Los locales y avisos iniciales son **datos de demostración** y los cambios viven en memoria mientras la app está abierta.
- **El radar es referencial:** representa los minutos a pie hasta cada local, no es un mapa geográfico.

1.3 Funcionalidades

Tabla 1. Funcionalidades del prototipo, la causa del problema que atacan y la pantalla que las contiene.

Funcionalidad	Causa que ataca	Pantalla
Buscar y filtrar menús por local, plato, precio o cocina	Información dispersa; difícil comparar	Menús
Ver precio, referencia, minutos a pie y platos de un menú	Información dispersa	Detalle
Ver y reportar si hay stock, quedan pocas raciones o se agotó	Disponibilidad sin aviso	Tarjeta, detalle y hoja inferior
Ver qué tan lejos está cada local en minutos a pie	Difícil comparar locales	Radar
Leer y publicar avisos de otros estudiantes	Disponibilidad sin aviso	Comunidad
Publicar un menú nuevo con vista previa	Información dispersa	Publicar menú

1.4 Herramientas y versiones

Tabla 2. Herramientas utilizadas y versiones comprobadas en el equipo de desarrollo.

Elemento	Detalle
Framework	Flutter 3.47.8 (canal stable)
Lenguaje	Dart 3.13.7
Diseño visual	Material 3 (`useMaterial3: true`) con tema propio
Paquetes externos en ejecución	Ninguno: solo el SDK de Flutter
Tipografías	Lilita One y Poppins, incluidas como recursos (licencia SIL OFL)
Dispositivo de prueba	Emulador Android «Pixel 9 Pro Dev» (Android 16, API 36)
Control de versiones	Git, con un commit por archivo modificado

2. Diseño de interfaces (funcionalidades y mockup)

Las cinco pantallas principales se muestran a continuación dentro de marcos de teléfono. Las capturas provienen del emulador Android, es decir, son la interfaz real construida con widgets y no maquetas dibujadas aparte.



Figura 2. Mockup general: Menús, Detalle, Radar, Comunidad y Publicar menú.

La navegación principal usa una barra inferior con tres destinos (**Menús**, **Radar** y **Comunidad**). Un botón flotante cambia según la pestaña: en Menús y Radar abre **Publicar menú**; en Comunidad abre **Nuevo aviso**. Desde una tarjeta o un pin se llega al **Detalle**.

2.1 Menús

Es la pantalla de entrada. Responde a la pregunta «¿dónde almuerzo hoy?» con el mínimo de texto: cada tarjeta tiene una **ilustración del plato típico de la cocina**, el precio en grande y el estado de disponibilidad sobre la imagen. Debajo se muestran el nombre, tres datos con ícono (minutos a pie, referencia y cocina) y los platos de fondo como chips del color de la cocina.

- **Indicador de menús libres:** cápsula azul con un punto verde que late como el eco de un radar y el número en la tipografía de titulares.
- **Búsqueda y filtros:** un campo de texto y chips con ícono para filtrar por precio o por cocina vegetariana.
- **Menú agotado:** la imagen se oscurece y aparece un sello «AGOTADO» inclinado, de modo que el estado se reconoce sin leer.



Figura 3. Pantalla Menús.



Figura 4. Menús al desplazar la lista.



Figura 5. Tarjeta de un menú agotado.

2.2 Detalle del menú

Al tocar una tarjeta, la ilustración «viaja» hasta la cabecera del detalle con una animación compartida. La cabecera se **colapsa al desplazar**, dejando solo el nombre del local. El contenido se organiza en tres bloques: nombre con precio y estado, tres fichas de datos clave y una tarjeta con la entrada, el plato de fondo y la bebida. Un botón fijo en la parte inferior permite avisar la disponibilidad.



Figura 6. Detalle con la cabecera expandida.



Figura 7. Detalle con la cabecera colapsándose.

2.3 Radar

El radar convierte la lista de locales en una imagen que se entiende de un vistazo. En el centro está el campus; los anillos marcan **2, 4 y 6 minutos a pie** y cada local se ubica a la distancia que le corresponde. Un barrido animado recorre el círculo y el punto central emite ondas, lo que le da el carácter de radar. Cada pin tiene el color del estado y el ícono de la cocina, y su precio debajo. Al tocarlo crece, brilla y muestra una tarjeta con la imagen del plato que lleva al detalle.



Figura 8. Radar sin selección.



Figura 9. Radar con un local seleccionado.

2.4 Comunidad

Comunidad es la pantalla con el diseño más libre. La cabecera tiene un borde ondulado, círculos translúcidos, una ilustración girada y un sticker amarillo inclinado con el número de avisos. Cada aviso es una **burbuja de conversación** que alterna de lado y cuyo color depende del estado. Lleva una miniatura circular del plato, un avatar con la inicial de quien publica y un sello de estado ligeramente girado.

¿Cómo se publica un aviso? Se evaluaron tres formas de identificar a quien publica:

Tabla 3. Opciones para identificar a quien publica un aviso en Comunidad.

Opción	Ventaja	Inconveniente	Decisión
Cuenta con correo institucional	Autores verificables	Exige servidor y registro; frena el primer aviso	Para una versión real
Apodo local, sin cuenta	Se publica en segundos y no requiere servidor	No hay verificación	Usada en el prototipo
Anónimo total	Cero fricción	Los avisos no tienen quién los respalde	Descartada

La primera vez, la hoja **Nuevo aviso** pide un apodo y lo recuerda mientras la app está abierta. Después solo pide tres datos: **dónde**, **cómo está** y un comentario opcional. Publicar un aviso también actualiza el estado del menú elegido, así que la lista y el radar reflejan lo que reportó la comunidad.



Figura 10. Comunidad con los avisos de ejemplo.



Figura 11. Comunidad tras publicar un aviso propio.



Figura 12. Hoja Nuevo aviso (primera vez, pide apodo).



Figura 13. Hoja Nuevo aviso con los datos completos.

2.5 Publicar menú

El formulario se diseñó para que no parezca largo ni complejo. Arriba hay una **vista previa en vivo** del banner: cambia al elegir la cocina y al escribir el nombre y el precio. Los campos se agrupan en tres secciones numeradas (*El local*, *El menú de hoy* y *¿Cuánto queda?*), cada campo tiene un ícono y un ejemplo, y los errores aparecen solo en el campo que se está llenando.



Figura 14. Formulario al abrir.



Figura 15. Vista previa con nombre y precio.



Figura 16. Cocina «Otro» seleccionada.

3. Descripción del proceso de desarrollo

3.1 Preparación del entorno

Se instaló el SDK de Flutter 3.47.8 y se agregó su carpeta `bin` al Path de usuario de Windows. `flutter doctor` confirmó el soporte para Chrome y Android. Con `flutter create --project-name menu_radar --platforms=android,web .` se generaron las carpetas de plataforma sobre el código ya escrito.

Durante la preparación aparecieron dos obstáculos que conviene dejar documentados. Primero, el emulador existente tenía su partición de datos de 6 GB casi llena (94 %), por lo que el APK de depuración de 143 MB **no se pudo instalar** («not enough space»). Se resolvió creando un emulador nuevo con 16 GB de almacenamiento. Segundo, el SDK de Android no tenía las `cmdline-tools`; esto no impide compilar, porque Gradle descargó y aceptó las licencias del NDK, Build-Tools y CMake durante la primera compilación.

3.2 Estructura del proyecto

Tabla 4. Organización de la carpeta lib/ y responsabilidad de cada archivo.

Archivo	Responsabilidad
`main.dart`	Arranque de la app y `MaterialApp` con el tema.
`theme/app_theme.dart`	Colores, tipografías y `ThemeData` (campos, botones, tarjetas, barra de navegación).
`models/menu.dart`	Clases `CampusMenu`, `CommunityNotice` y el estado de disponibilidad.
`data/menu_store.dart`	Estado en memoria (`ChangeNotifier`): menús, avisos y apodo de sesión.
`screens/home_shell.dart`	`Scaffold`, `NavigationBar`, `IndexedStack` y botón flotante.
`screens/explore_tab.dart`	Pestaña Menús: encabezado, búsqueda, filtros y lista.
`screens/radar_tab.dart`	Pestaña Radar: lienzo animado, pines y vista previa.
`screens/community_tab.dart`	Pestaña Comunidad: cabecera ondulada y burbujas.
`screens/menu_detail_page.dart`	Detalle con `SliverAppBar` colapsable.
`screens/publish_menu_page.dart`	Formulario de publicación con vista previa.
`widgets/food_art.dart`	Ilustraciones de platos (`CustomPainter`) y banner por cocina.
`widgets/menu_card.dart`	Tarjeta de menú de la lista.
`widgets/status_badge.dart`	Indicador de disponibilidad.
`widgets/availability_sheet.dart`	Hoja inferior para reportar stock.
`widgets/notice_composer.dart`	Hoja inferior para publicar un aviso.

3.3 Cómo Flutter organiza el layout

Todo en Flutter es un widget, y el layout se resuelve con una regla simple: **las restricciones bajan, los tamaños suben y el padre decide la posición**. Cada widget recibe del padre un rango de tamaños permitido (sus restricciones), elige un tamaño dentro de ese rango y se lo comunica al padre, que lo ubica. Entender esto explica la mayoría de los errores de desborde. Por ejemplo, una `ListView` dentro de una `Column` falla si no se le acota la altura, y por eso en el proyecto siempre va dentro de un `Expanded`.

Tabla 5. Widgets de organización y layout estudiados, su función y dónde se usaron.

Widget	Función	Uso en MenúRadar
`Scaffold`	Estructura base: cuerpo, barra de navegación y botón flotante.	Todas las pantallas.
`SafeArea`	Evita invadir barras del sistema y recortes.	Shell y detalle.
`Column` / `Row`	Ordenan hijos en vertical u horizontal.	Encabezados, tarjetas, fichas de datos.
`Expanded`	Reparte el espacio libre del eje principal.	Lista de menús, panel del radar, fichas.
`Padding` / `SizedBox`	Márgenes y separaciones fijas.	Toda la app.
`Wrap`	Pasa a otra línea cuando falta ancho.	Datos y chips de la tarjeta; leyenda.
`ListView`	Contenido desplazable.	Menús, Comunidad, formulario, filtros.
`CustomScrollView` / `SliverAppBar`	Cabecera que se colapsa con el scroll.	Detalle del menú.
`Stack` / `Positioned`	Superponen y ubican elementos.	Banner de tarjeta, radar, cabecera.
`LayoutBuilder`	Entrega las restricciones recibidas para calcular medidas.	Posición de los pines del radar.
`IndexedStack`	Muestra un hijo y conserva el estado de los demás.	Pestañas del shell.
`NavigationBar`	Navegación principal de Material 3.	Menús, Radar, Comunidad.
`Hero`	Transición compartida entre pantallas.	Banner de la tarjeta al detalle.
`ClipRRect` / `ClipOval` / `ClipPath`	Recortan en formas.	Banner, miniaturas, onda de Comunidad.
`Transform.rotate`	Gira un widget.	Sellos y sticker.
`CustomPaint`	Dibujo propio sobre un lienzo.	Ilustraciones y radar.
`AnimatedSwitcher` / `AnimatedScale`	Animaciones implícitas.	Vista previa y pines del radar.

3.4 Construcción de cada pantalla

Menús

La pestaña es una `Column`: el encabezado y el buscador ocupan su alto natural, la fila de filtros es una `ListView` horizontal de altura fija, y la lista de tarjetas está dentro de un `Expanded` para recibir el espacio restante. Cada tarjeta es un `Card` con `InkWell`, de modo que toda la superficie es táctil.

El banner usa un `Stack`: la ilustración va al fondo y encima, con `Positioned`, el estado arriba a la izquierda y el precio abajo.

```
Scaffold
└─ SafeArea
   └─ IndexedStack
      └─ ExploreTab · Column
         ├─ Row (logo, título, indicador de libres)
         ├─ TextField
         ├─ ListView horizontal · FilterChip
         └─ Expanded
            └─ ListView.separated
               └─ MenuCard · Card
                  └─ InkWell · Column
                     ├─ Stack (Hero > FoodBanner, StatusBadge, precio)
                     └─ Padding (nombre, Wrap de datos, Wrap de chips)
```

Detalle

El detalle usa `CustomScrollView` con dos slivers. El `SliverAppBar` es `pinned` y tiene `expandedHeight`; dentro de su `flexibleSpace`, un `LayoutBuilder` calcula cuánto se ha colapsado para cambiar la opacidad del título. El contenido va en un `SliverToBoxAdapter`. Las tres fichas de datos son un `Row` con tres `Expanded`, por lo que reparten el ancho en partes iguales.

```
Scaffold
├─ body: CustomScrollView
│  ├─ SliverAppBar (pinned, expandedHeight 280)
│  │  └─ flexibleSpace: LayoutBuilder → Stack
│  │     ├─ Hero → FoodBanner (CustomPaint)
│  │     ├─ Opacity → Text (título al colapsar)
│  │     └─ Container (borde superior redondeado)
│  └─ SliverToBoxAdapter → Padding → Column
│     ├─ Text (nombre), Row (precio, StatusBadge)
│     ├─ Row → 3 × Expanded → _StatTile
│     ├─ Card → Column → 3 × _CourseRow
│     └─ Row (ícono + aviso)
└─ bottomNavigationBar: SafeArea → FilledButton.icon
```

Radar

El radar es un `StatefulWidget` con un `AnimationController` que dura cinco segundos y se repite. Un `LayoutBuilder` entrega el tamaño disponible; sobre él, un `Stack` superpone el lienzo (`CustomPaint`), el marcador del campus y un `Positioned` por cada local. Como el painter recibe la animación como `repaint`, redibuja el barrido en cada cuadro sin reconstruir los demás widgets.

```
RadarTab (StatefulWidget + SingleTickerProviderStateMixin)
└─ Padding → Column
   ├─ Row (títulos, cápsula con el número de locales)
   └─ Expanded → ClipRRect → LayoutBuilder → GestureDetector → Stack
      ├─ Positioned.fill → CustomPaint(_RadarPainter)
      ├─ Positioned → _CampusMarker
      ├─ Positioned × n → _RadarPin
      ├─ Positioned → _Legend (Wrap)
      └─ Positioned → AnimatedSwitcher → _Hint | _PreviewCard
```

La posición de cada pin se calcula con una fórmula propia: el **ángulo** sale de la posición ilustrativa del menú y la **distancia al centro** es proporcional a los minutos a pie. Se deja un radio mínimo para que ningún pin tape el marcador del campus. Los anillos del lienzo usan la misma función, de modo que pines y anillos siempre coinciden.

```
/// Fracción del radio del radar donde cae un local según sus minutos a pie.
/// Deja un espacio libre alrededor del marcador del campus.
double radarFraction(num minutes) =>
    0.22 + 0.78 * minutes.clamp(1, radarMaxMinutes) / radarMaxMinutes;
```

```
/// La distancia al centro equivale a los minutos a pie; el ángulo sale de la
/// posición ilustrativa del menú.
Offset _pinCenter(CampusMenu menu, Size size) {
  final angle = math.atan2(menu.positionY - 0.5, menu.positionX - 0.5);
```

```
    return size.center(Offset.zero) +
        Offset(math.cos(angle), math.sin(angle)) *
            (radarFraction(menu.walkMinutes) * _radarRadius(size));
}
```

Comunidad

La pantalla es un `ListView`. Su cabecera combina un `ClipPath` con un `CustomClipper` que recorta la onda inferior, un degradado, círculos dibujados con `CustomPaint`, una ilustración girada con `Transform.rotate` y un sticker. Las burbujas alternan el orden de sus hijos en un `Row` y la esquina redondeada cambia de lado según la posición.

```
class _WaveClipper extends CustomClipper<Path> {
  @override
  Path getClip(Size size) => Path()
    ..lineTo(0, size.height - 36)
    ..quadraticBezierTo(size.width * 0.25, size.height + 8, size.width * 0.52, size.height - 24)
    ..quadraticBezierTo(size.width * 0.80, size.height - 54, size.width, size.height - 14)
    ..lineTo(size.width, 0)
    ..close();

  @override
  bool shouldReclip(covariant CustomClipper<Path> oldClipper) => false;
}
```

Publicar menú

El formulario es un `Form` con un `ListView`. Cada campo es un `TextFormField` con `prefixIcon`, y un método auxiliar `_field` evita repetir la configuración. La vista previa escucha los controladores de nombre y precio con `ListenableBuilder`, así que se actualiza mientras se escribe. El selector de cocina son `ChoiceChip` dentro de una lista horizontal, y la disponibilidad un `SegmentedButton` dentro de un `Expanded` para que ocupe todo el ancho.

```
Scaffold → SafeArea → Form → ListView
├─ _Preview: SizedBox → FoodBanner → ListenableBuilder → Column
├─ SizedBox → ListView horizontal → ChoiceChip
├─ _Section(1) → TextFormField ×3 + Row[Expanded TextFormField ×2]
├─ _Section(2) → TextFormField ×3
├─ _Section(3) → Row → Expanded → SegmentedButton
└─ FilledButton.icon
```

La validación es por campo: cada `TextFormField` usa `AutovalidateMode.onUserInteraction`, de modo que un campo solo se marca en rojo cuando el usuario interactúa con él. Al pulsar *Publicar menú* se valida el formulario completo. Para que un menú nuevo no se encime con otro en el radar, `_freeAngle` prueba 24 ángulos y elige el más alejado de los pines que ya existen.

3.5 Tipografía y tema

Se incluyeron dos familias libres como recursos del proyecto, para que la app no dependa de internet. `ThemeData(fontFamily: ...)` fija Poppins como base y los estilos de títulos del tema usan Lilita One. Como Lilita One tiene un solo peso, nunca se le aplica negrita (la sintética se ve deformada); un método auxiliar `AppText.display()` entrega el estilo de titular.

Tabla 6. Tipografías del proyecto.

Familia	Uso	Motivo
Lilita One	Títulos, precios, nombres y sellos	Trazo grueso y redondeado, afín a un letrero de comida; da identidad con pocas palabras.
Poppins (Regular, Medium, SemiBold, Bold)	Lectura, etiquetas y botones	Geométrica y muy legible en pantallas pequeñas.

```
  /// Estilo de titular. Lilita One solo tiene un peso, por eso no se
  /// le asigna negrita.
  static TextStyle display(
    double size, {
    Color color = AppColors.navy,
    double height = 1.1,
    double letterSpacing = 0,
    List<Shadow>? shadows,
  }) => TextStyle(
    fontFamily: displayFamily,
    fontSize: size,
    color: color,
    height: height,
    letterSpacing: letterSpacing,
    fontWeight: FontWeight.w400,
    shadows: shadows,
  );
```

3.6 Ilustraciones con CustomPainter

Para que las tarjetas fueran visuales sin depender de fotos con licencia, los platos se dibujan con `CustomPainter` en un lienzo normalizado de 0 a 1 (`lib/widgets/food_art.dart`). Al escalar el canvas por el tamaño del widget, la misma ilustración sirve para el banner de una tarjeta, la cabecera del detalle o una miniatura de 56 píxeles. Cada cocina tiene su paleta de degradado y su plato.



Figura 17. Las cuatro ilustraciones: plato criollo, bowl vegetariano, bowl de chifa y cloche genérica.

Tabla 7. Ilustración de cada cocina y las formas con que se dibuja.

Cocina	Elementos dibujados
Criollo	Plato circular con arroz (óvalo con granos), papas fritas y tiras de carne (rectángulos redondeados girados), cebolla (arcos) y tomate.
Vegetariano	Bowl formado por una elipse y un arco, hojas (elipses giradas), tomates y zanahoria (círculos).
Chifa	Bowl rojo, fideos (curvas de Bézier), cebollín (círculos), palillos (líneas) y vapor.
Otro	Cloche: semicírculo con brillo, base, perilla y destellos en cruz.

```
void _bar(
  Canvas canvas,
  Offset center,
  double width,
  double height,
```

```
  double angle,
  Color color,
) {
  canvas
    ..save()
    ..translate(center.dx, center.dy)
    ..rotate(angle)
    ..drawRRect(
      RRect.fromRectAndRadius(
        Rect.fromCenter(center: Offset.zero, width: width, height: height),
        Radius.circular(math.min(width, height) * 0.45),
      ),
      _fill(color),
    )
    ..restore();
}
```

3.7 Pruebas y control de versiones

Tabla 8. Verificaciones realizadas sobre el proyecto.

Verificación	Resultado
`flutter analyze`	Sin avisos ni errores.
`flutter test`	Prueba de widget superada: la app arranca y muestra la navegación.
`flutter build apk --debug`	Compila (APK de 143 MB).
`flutter build apk --release --target-platform android-x64`	Compila (APK de 17,8 MB).
Ejecución en el emulador	Se recorrieron las cinco pantallas, el reporte de disponibilidad, la publicación de un aviso y la validación del formulario.

El código se versionó con Git haciendo **un commit por archivo modificado**, con mensajes que describen el cambio de ese archivo. Esto deja un historial de más de cien commits que permite seguir cómo evolucionó cada pantalla.

4. Apreciación de la herramienta

Tras construir el prototipo, estas son dos características de Flutter a favor y dos en contra, basadas en lo que ocurrió durante el desarrollo.

4.1 Características a favor

1. Ciclo de desarrollo rápido gracias a Hot Reload. Se pudieron probar cambios de layout, colores y tipografía en el emulador casi al instante, sin reiniciar la app. Esto fue decisivo en el rediseño, donde se ajustaron márgenes, tamaños de letra y posiciones de pines varias veces hasta que la pantalla se vio como se esperaba.

2. Control total sobre la interfaz. Flutter dibuja sus propios widgets en lugar de apoyarse en los controles nativos, y ofrece `CustomPainter` para dibujar cualquier forma. Con eso se crearon las ilustraciones de los platos, el radar con su barrido animado y la onda de Comunidad sin imágenes externas ni librerías, y el resultado se ve igual en cualquier dispositivo.

4.2 Características en contra

1. Código muy anidado y extenso. Una interfaz se describe como un árbol de widgets, y una pantalla sencilla puede superar los diez niveles de sangría. Un archivo como `radar_tab.dart` llega a más de 450 líneas, y para mantenerlo legible hubo que dividir las pantallas en widgets privados. Esto exige disciplina y cierta práctica para no perderse en el árbol.

2. Entorno de desarrollo pesado. Preparar Flutter para Android exige el SDK, el emulador, las licencias y la configuración de Gradle, y consume mucho espacio y tiempo. En el proyecto, un emulador sin espacio impidió instalar el APK de depuración de 143 MB, y la primera compilación tardó varios minutos. El tamaño de la app de depuración es también mucho mayor que el de la versión final (17,8 MB).

Anexos: código de la aplicación

Se incluye el código fuente completo de la aplicación (carpeta `lib/`), el archivo `pubspec.yaml` y la prueba de widget. Los recursos tipográficos y las capturas se encuentran en el repositorio del proyecto.

Anexo A. pubspec.yaml

```
name: menu_radar
description: Prototipo multiplataforma de interfaces para consultar menus universitarios.
publish_to: none
version: 1.0.0+1

environment:
  sdk: '>=3.6.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true

  fonts:
    - family: LilitaOne
      fonts:
        - asset: assets/fonts/LilitaOne-Regular.ttf
    - family: Poppins
      fonts:
        - asset: assets/fonts/Poppins-Regular.ttf
        - asset: assets/fonts/Poppins-Medium.ttf
          weight: 500
        - asset: assets/fonts/Poppins-SemiBold.ttf
          weight: 600
        - asset: assets/fonts/Poppins-Bold.ttf
          weight: 700
```

Anexo B. lib/main.dart

```
import 'package:flutter/material.dart';

import 'data/menu_store.dart';
import 'screens/home_shell.dart';
import 'theme/app_theme.dart';

void main() => runApp(const MenuRadarApp());

class MenuRadarApp extends StatefulWidget {
  const MenuRadarApp({super.key});

  @override
  State<MenuRadarApp> createState() => _MenuRadarAppState();
}

class _MenuRadarAppState extends State<MenuRadarApp> {
  final MenuStore _store = MenuStore();

  @override
  void dispose() {
    _store.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'MenúRadar',
    debugShowCheckedModeBanner: false,
```

```
    theme: buildAppTheme(),
    home: HomeShell(store: _store),
  );
}
```

Anexo C. lib/theme/app_theme.dart

```
import 'package:flutter/material.dart';

class AppColors {
  static const orange = Color(0xFFDA4B0B);
  static const cream = Color(0xFFFFFBF4);
  static const navy = Color(0xFF14233A);
  static const green = Color(0xFF087C63);
  static const amber = Color(0xFF9A5D00);
  static const red = Color(0xFFB33434);
  static const sun = Color(0xFFFFC145);
  static const ember = Color(0xFFB8360A);
}

/// Tipografías: Poppins para el texto y Lilita One para títulos y precios.
class AppText {
  static const body = 'Poppins';
  static const displayFamily = 'LilitaOne';

  /// Estilo de titular. Lilita One solo tiene un peso, por eso no se
  /// le asigna negrita.
  static TextStyle display(
    double size, {
    Color color = AppColors.navy,
    double height = 1.1,
    double letterSpacing = 0,
    List<Shadow>? shadows,
  }) => TextStyle(
    fontFamily: displayFamily,
    fontSize: size,
    color: color,
    height: height,
    letterSpacing: letterSpacing,
    fontWeight: FontWeight.w400,
    shadows: shadows,
  );
}

ThemeData buildAppTheme() {
  final scheme = ColorScheme.fromSeed(
    seedColor: AppColors.orange,
    primary: AppColors.orange,
    surface: AppColors.cream,
  );
  final base = ThemeData(
    useMaterial3: true,
    colorScheme: scheme,
    fontFamily: AppText.body,
  );
  final text = base.textTheme;
  return base.copyWith(
    scaffoldBackgroundColor: AppColors.cream,
    textTheme: text.copyWith(
      headlineMedium: text.headlineMedium?.copyWith(
        fontFamily: AppText.displayFamily,
        fontWeight: FontWeight.w400,
      ),
      headlineSmall: text.headlineSmall?.copyWith(
        fontFamily: AppText.displayFamily,
        fontWeight: FontWeight.w400,
      ),
      titleLarge: text.titleLarge?.copyWith(
        fontFamily: AppText.displayFamily,
        fontWeight: FontWeight.w400,
      ),
    ),
    appBarTheme: const AppBarTheme(
      backgroundColor: AppColors.cream,
      foregroundColor: AppColors.navy,
      centerTitle: false,
      titleTextStyle: TextStyle(
        fontFamily: AppText.displayFamily,
        fontSize: 22,
        color: AppColors.navy,
      ),
```

```
    ),
    cardTheme: CardThemeData(
      color: Colors.white,
      elevation: 0,
      shape: RoundedRectangleBorder(
        side: const BorderSide(color: Color(0xFFEDE5DA)),
        borderRadius: BorderRadius.circular(20),
      ),
    ),
    inputDecorationTheme: InputDecorationThemeData(
      filled: true,
      fillColor: Colors.white,
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(16),
        borderSide: const BorderSide(color: Color(0xFFE6DCCD)),
      ),
      enabledBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(16),
        borderSide: const BorderSide(color: Color(0xFFE6DCCD)),
      ),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(16),
        borderSide: const BorderSide(color: AppColors.orange, width: 2),
      ),
      prefixIconColor: AppColors.orange,
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
    ),
    filledButtonTheme: FilledButtonThemeData(
      style: FilledButton.styleFrom(
        minimumSize: const Size.fromHeight(54),
        textStyle: const TextStyle(
          fontFamily: AppText.body,
          fontWeight: FontWeight.w600,
          fontSize: 16,
        ),
      ),
    ),
    navigationBarTheme: NavigationBarThemeData(
      backgroundColor: Colors.white,
      indicatorColor: const Color(0xFFFFE9D8),
      labelTextStyle: WidgetStateProperty.all(
        const TextStyle(fontWeight: FontWeight.w600, fontSize: 12),
      ),
    ),
  );
}
```

Anexo D. lib/models/menu.dart

```
enum MenuAvailability { available, limited, soldOut }

extension MenuAvailabilityLabel on MenuAvailability {
  String get label => switch (this) {
    MenuAvailability.available => 'Disponible',
    MenuAvailability.limited => 'Últimas raciones',
    MenuAvailability.soldOut => 'Agotado',
  };

  String get noticeText => switch (this) {
    MenuAvailability.available => 'Hay raciones disponibles.',
    MenuAvailability.limited => 'Quedan pocas raciones.',
    MenuAvailability.soldOut => 'Se agotó por ahora.',
  };
}

/// Minutos a pie que representa el anillo exterior del radar.
const int radarMaxMinutes = 6;

/// Fracción del radio del radar donde cae un local según sus minutos a pie.
/// Deja un espacio libre alrededor del marcador del campus.
double radarFraction(num minutes) =>
    0.22 + 0.78 * minutes.clamp(1, radarMaxMinutes) / radarMaxMinutes;

class CampusMenu {
  const CampusMenu({
    required this.id,
    required this.restaurant,
    required this.price,
    required this.walkMinutes,
    required this.area,
    required this.category,
```

```
    required this.starters,
    required this.mainDishes,
    required this.drink,
    required this.availability,
    required this.positionX,
    required this.positionY,
  });

  final String id;
  final String restaurant;
  final double price;
  final int walkMinutes;
  final String area;
  final String category;
  final List<String> starters;
  final List<String> mainDishes;
  final String drink;
  final MenuAvailability availability;
  final double positionX;
  final double positionY;

  CampusMenu copyWith({MenuAvailability? availability}) => CampusMenu(
    id: id,
    restaurant: restaurant,
    price: price,
    walkMinutes: walkMinutes,
    area: area,
    category: category,
    starters: starters,
    mainDishes: mainDishes,
    drink: drink,
    availability: availability ?? this.availability,
    positionX: positionX,
    positionY: positionY,
  );
}

class CommunityNotice {
  const CommunityNotice({
    required this.author,
    required this.restaurant,
    required this.category,
    required this.message,
    required this.availability,
    required this.createdAt,
  });

  final String author;
  final String restaurant;
  final String category;
  final String message;
  final MenuAvailability availability;
  final DateTime createdAt;
}
```

Anexo E. lib/data/menu_store.dart

```
import 'package:flutter/foundation.dart';

import '../models/menu.dart';

class MenuStore extends ChangeNotifier {
  final List<CampusMenu> _menus = [
    const CampusMenu(
      id: '1',
      restaurant: 'La Tía Rosa',
      price: 9.50,
      walkMinutes: 2,
      area: 'Puerta 3',
      category: 'Criollo',
      starters: ['Papa a la huancaína', 'Sopa del día'],
      mainDishes: ['Lomo saltado', 'Ají de gallina'],
      drink: 'Chicha morada',
      availability: MenuAvailability.available,
      positionX: 0.30,
      positionY: 0.32,
    ),
    const CampusMenu(
      id: '2',
      restaurant: 'Sazón de Casa',
```

```
    price: 11,
    walkMinutes: 4,
    area: 'Av. Independencia',
    category: 'Vegetariano',
    starters: ['Ensalada fresca'],
    mainDishes: ['Tallarines verdes', 'Guiso de lentejas'],
    drink: 'Limonada',
    availability: MenuAvailability.limited,
    positionX: 0.66,
    positionY: 0.28,
  ),
  const CampusMenu(
    id: '3',
    restaurant: 'Chifa Campus',
    price: 12,
    walkMinutes: 5,
    area: 'Puerta 1',
    category: 'Chifa',
    starters: ['Sopa wantan'],
    mainDishes: ['Arroz chaufa', 'Tallarín saltado'],
    drink: 'Refresco del día',
    availability: MenuAvailability.available,
    positionX: 0.18,
    positionY: 0.68,
  ),
  const CampusMenu(
    id: '4',
    restaurant: 'El Buen Sabor',
    price: 10,
    walkMinutes: 3,
    area: 'Puerta 3',
    category: 'Criollo',
    starters: ['Caldo de verduras'],
    mainDishes: ['Pollo al horno', 'Seco de res'],
    drink: 'Emoliente',
    availability: MenuAvailability.soldOut,
    positionX: 0.77,
    positionY: 0.62,
  ),
];

final List<CommunityNotice> _notices = [
  CommunityNotice(
    author: 'Valeria R.',
    restaurant: 'Sazón de Casa',
    category: 'Vegetariano',
    message: 'Quedan pocas raciones.',
    availability: MenuAvailability.limited,
    createdAt: DateTime.now().subtract(const Duration(minutes: 12)),
  ),
  CommunityNotice(
    author: 'Carlos M.',
    restaurant: 'El Buen Sabor',
    category: 'Criollo',
    message: 'Se agotó por hoy.',
    availability: MenuAvailability.soldOut,
    createdAt: DateTime.now().subtract(const Duration(minutes: 25)),
  ),
];

/// Apodo de quien publica. No hay cuentas: se elige la primera vez que se
/// publica un aviso y dura mientras la app está abierta.
String? _alias;

String? get alias => _alias;

String get _author => _alias ?? 'Anónimo';

void setAlias(String value) {
  _alias = value.trim();
  notifyListeners();
}

List<CampusMenu> get menus => List.unmodifiable(_menus);
List<CommunityNotice> get notices => List.unmodifiable(_notices);
int get availableCount => _menus
    .where((menu) => menu.availability != MenuAvailability.soldOut)
    .length;

CampusMenu findMenu(String id) => _menus.firstWhere((menu) => menu.id == id);

/// Actualiza el estado de un menú y publica un aviso. Si [comment] viene
/// vacío se usa el texto estándar del estado.
```

```
  void updateAvailability(
    String id,
    MenuAvailability availability, {
    String? comment,
  }) {
    final index = _menus.indexWhere((menu) => menu.id == id);
    final text = comment?.trim() ?? '';
    if (index < 0) return;
    if (_menus[index].availability == availability && text.isEmpty) return;
    final current = _menus[index];
    _menus[index] = current.copyWith(availability: availability);
    _notices.insert(
      0,
      CommunityNotice(
        author: _author,
        restaurant: current.restaurant,
        category: current.category,
        message: text.isEmpty ? availability.noticeText : text,
        availability: availability,
        createdAt: DateTime.now(),
      ),
    );
    notifyListeners();
  }

  void addMenu(CampusMenu menu) {
    _menus.insert(0, menu);
    _notices.insert(
      0,
      CommunityNotice(
        author: _author,
        restaurant: menu.restaurant,
        category: menu.category,
        message: 'Publicó un menú por S/ ${menu.price.toStringAsFixed(2)}.',
        availability: menu.availability,
        createdAt: DateTime.now(),
      ),
    );
    notifyListeners();
  }
}
```

Anexo F. lib/screens/home_shell.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../widgets/availability_sheet.dart';
import '../widgets/notice_composer.dart';
import 'community_tab.dart';
import 'explore_tab.dart';
import 'menu_detail_page.dart';
import 'publish_menu_page.dart';
import 'radar_tab.dart';

class HomeShell extends StatefulWidget {
  const HomeShell({super.key, required this.store});

  final MenuStore store;

  @override
  State<HomeShell> createState() => _HomeShellState();
}

class _HomeShellState extends State<HomeShell> {
  int _selectedIndex = 0;

  Future<void> _openMenu(CampusMenu menu) async {
    await Navigator.push<void>(
      context,
      MaterialPageRoute(
        builder: (_) => MenuDetailPage(store: widget.store, menuId: menu.id),
      ),
    );
  }

  Future<void> _publishMenu() async {
    final id = await Navigator.push<String>(
```

```
      context,
      MaterialPageRoute(builder: (_) => PublishMenuPage(store: widget.store)),
    );
    if (id == null || !mounted) return;
    setState(() => _selectedIndex = 0);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Menú publicado en esta sesión.')),
    );
  }

  Future<void> _newNotice() async {
    final published = await showNoticeComposer(context, widget.store);
    if (!published || !mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Aviso publicado en esta sesión.')),
    );
  }

  @override
  Widget build(BuildContext context) => Scaffold(
    // Comunidad dibuja su cabecera hasta el borde superior de la pantalla,
    // por eso allí la barra de estado lleva íconos claros.
    body: AnnotatedRegion<SystemUiOverlayStyle>(
      value: (_selectedIndex == 2
              ? SystemUiOverlayStyle.light
              : SystemUiOverlayStyle.dark)
          .copyWith(statusBarColor: Colors.transparent),
      child: SafeArea(
        top: _selectedIndex != 2,
        child: IndexedStack(
          index: _selectedIndex,
          children: [
            ExploreTab(
              store: widget.store,
              onOpen: _openMenu,
              onReport: (menu) => reportAvailability(context, widget.store, menu),
            ),
            RadarTab(store: widget.store, onOpen: _openMenu),
            CommunityTab(store: widget.store),
          ],
        ),
      ),
    ),
    floatingActionButton: FloatingActionButton.extended(
      onPressed: _selectedIndex == 2 ? _newNotice : _publishMenu,
      icon: Icon(_selectedIndex == 2 ? Icons.edit_note_rounded : Icons.add_rounded),
      label: Text(_selectedIndex == 2 ? 'Nuevo aviso' : 'Publicar menú'),
    ),
    bottomNavigationBar: NavigationBar(
      selectedIndex: _selectedIndex,
      onDestinationSelected: (index) => setState(() => _selectedIndex = index),
      destinations: const [
        NavigationDestination(
          icon: Icon(Icons.restaurant_menu_outlined),
          selectedIcon: Icon(Icons.restaurant_menu),
          label: 'Menús',
        ),
        NavigationDestination(
          icon: Icon(Icons.radar_outlined),
          selectedIcon: Icon(Icons.radar),
          label: 'Radar',
        ),
        NavigationDestination(
          icon: Icon(Icons.forum_outlined),
          selectedIcon: Icon(Icons.forum),
          label: 'Comunidad',
        ),
      ],
    ),
  );
}
```

Anexo G. lib/screens/explore_tab.dart

```
import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../theme/app_theme.dart';
import '../widgets/menu_card.dart';
```

```
class ExploreTab extends StatefulWidget {
  const ExploreTab({
    super.key,
    required this.store,
    required this.onOpen,
    required this.onReport,
  });

  final MenuStore store;
  final ValueChanged<CampusMenu> onOpen;
  final ValueChanged<CampusMenu> onReport;

  @override
  State<ExploreTab> createState() => _ExploreTabState();
}

class _ExploreTabState extends State<ExploreTab> {
  final TextEditingController _search = TextEditingController();
  String _filter = 'Todos';

  static const _filters = [
    ('Todos', Icons.apps_rounded),
    ('Hasta S/ 10', Icons.savings_outlined),
    ('S/ 11-14', Icons.payments_outlined),
    ('Vegetariano', Icons.eco_outlined),
  ];

  @override
  void dispose() {
    _search.dispose();
    super.dispose();
  }

  bool _matches(CampusMenu menu) {
    final query = _search.text.trim().toLowerCase();
    final text = [
      menu.restaurant,
      menu.category,
      ...menu.starters,
      ...menu.mainDishes,
    ].join(' ').toLowerCase();
    if (query.isNotEmpty && !text.contains(query)) return false;
    return switch (_filter) {
      'Hasta S/ 10' => menu.price <= 10,
      'S/ 11-14' => menu.price >= 11 && menu.price <= 14,
      'Vegetariano' => menu.category == 'Vegetariano',
      _ => true,
    };
  }

  @override
  Widget build(BuildContext context) => AnimatedBuilder(
    animation: widget.store,
    builder: (context, _) {
      final menus = widget.store.menus.where(_matches).toList();
      return Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Padding(
            padding: const EdgeInsets.fromLTRB(20, 16, 20, 0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  children: [
                    Container(
                      width: 46,
                      height: 46,
                      decoration: BoxDecoration(
                        color: AppColors.orange,
                        borderRadius: BorderRadius.circular(15),
                      ),
                      child: const Icon(Icons.ramen_dining_rounded, color: Colors.white),
                    ),
                    const SizedBox(width: 12),
                    Expanded(
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            'MenúRadar',
                            style: Theme.of(context).textTheme.titleLarge?.copyWith(
                              color: AppColors.navy,
                              fontSize: 28,
```

```
                            height: 1.1,
                          ),
                        ),
                        Text(
                          '¿Dónde almuerzas hoy?',
                          style: Theme.of(context).textTheme.bodyMedium,
                        ),
                      ],
                    ),
                  ),
                  _AvailablePill(count: widget.store.availableCount),
                ],
              ),
              const SizedBox(height: 16),
              TextField(
                controller: _search,
                onChanged: (_) => setState(() {}),
                decoration: const InputDecoration(
                  hintText: 'Buscar local o plato',
                  prefixIcon: Icon(Icons.search_rounded),
                ),
              ),
            ],
          ),
        ),
        const SizedBox(height: 12),
        SizedBox(
          height: 44,
          child: ListView.separated(
            scrollDirection: Axis.horizontal,
            padding: const EdgeInsets.symmetric(horizontal: 20),
            itemCount: _filters.length,
            separatorBuilder: (context, index) => const SizedBox(width: 8),
            itemBuilder: (context, index) {
              final (label, icon) = _filters[index];
              return FilterChip(
                avatar: Icon(icon, size: 18),
                label: Text(label),
                showCheckmark: false,
                selected: _filter == label,
                onSelected: (_) => setState(() => _filter = label),
              );
            },
          ),
        ),
        const SizedBox(height: 12),
        Expanded(
          child: menus.isEmpty
              ? const _EmptyState()
              : ListView.separated(
                  padding: const EdgeInsets.fromLTRB(20, 0, 20, 100),
                  itemCount: menus.length,
                  separatorBuilder: (context, index) => const SizedBox(height: 16),
                  itemBuilder: (context, index) => MenuCard(
                    menu: menus[index],
                    onOpen: () => widget.onOpen(menus[index]),
                    onReport: () => widget.onReport(menus[index]),
                  ),
                ),
        ),
      ],
    );
  },
);
}

/// Indicador de menús libres: un punto que late como un eco de radar junto al
/// número, en la tipografía de titulares.
class _AvailablePill extends StatefulWidget {
  const _AvailablePill({required this.count});

  final int count;

  @override
  State<_AvailablePill> createState() => _AvailablePillState();
}

class _AvailablePillState extends State<_AvailablePill>
    with SingleTickerProviderStateMixin {
  late final AnimationController _pulse = AnimationController(
    vsync: this,
    duration: const Duration(milliseconds: 1800),
  )..repeat();
```

```
  @override
  void dispose() {
    _pulse.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => Container(
    padding: const EdgeInsets.fromLTRB(8, 6, 14, 6),
    decoration: BoxDecoration(
      color: AppColors.navy,
      borderRadius: const BorderRadius.only(
        topLeft: Radius.circular(22),
        topRight: Radius.circular(22),
        bottomRight: Radius.circular(22),
        bottomLeft: Radius.circular(6),
      ),
      boxShadow: [
        BoxShadow(
          color: AppColors.navy.withAlpha(60),
          blurRadius: 10,
          offset: const Offset(0, 4),
        ),
      ],
    ),
    child: Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        SizedBox(
          width: 28,
          height: 28,
          child: CustomPaint(painter: _EchoPainter(_pulse)),
        ),
        const SizedBox(width: 4),
        Text(
          '${widget.count}',
          style: AppText.display(28, color: AppColors.sun, height: 1),
        ),
        const SizedBox(width: 6),
        const Text(
          'menús\nlibres',
          style: TextStyle(
            color: Colors.white70,
            fontSize: 11,
            height: 1.15,
            fontWeight: FontWeight.w600,
          ),
        ),
      ],
    ),
  );
}

class _EchoPainter extends CustomPainter {
  _EchoPainter(this.animation) : super(repaint: animation);

  final Animation<double> animation;

  @override
  void paint(Canvas canvas, Size size) {
    const dot = Color(0xFF3DDC97);
    final center = size.center(Offset.zero);
    final t = animation.value;
    canvas
      ..drawCircle(
        center,
        4 + t * 10,
        Paint()
          ..color = dot.withAlpha(((1 - t) * 180).round())
          ..style = PaintingStyle.stroke
          ..strokeWidth = 1.6,
      )
      ..drawCircle(center, 4.5, Paint()..color = dot);
  }

  @override
  bool shouldRepaint(covariant _EchoPainter oldDelegate) => false;
}

class _EmptyState extends StatelessWidget {
  const _EmptyState();

  @override
  Widget build(BuildContext context) => const Center(
```

```
    child: Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Icon(Icons.search_off_rounded, size: 56, color: Colors.black26),
        SizedBox(height: 8),
        Text('Sin resultados'),
      ],
    ),
  );
}
```

Anexo H. lib/screens/radar_tab.dart

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../theme/app_theme.dart';
import '../widgets/food_art.dart';
import '../widgets/status_badge.dart';

double _radarRadius(Size size) =>
    math.max(40, math.min(size.width, size.height) / 2 - 36);

/// La distancia al centro equivale a los minutos a pie; el ángulo sale de la
/// posición ilustrativa del menú.
Offset _pinCenter(CampusMenu menu, Size size) {
  final angle = math.atan2(menu.positionY - 0.5, menu.positionX - 0.5);
  return size.center(Offset.zero) +
      Offset(math.cos(angle), math.sin(angle)) *
          (radarFraction(menu.walkMinutes) * _radarRadius(size));
}

class RadarTab extends StatefulWidget {
  const RadarTab({super.key, required this.store, required this.onOpen});

  final MenuStore store;
  final ValueChanged<CampusMenu> onOpen;

  @override
  State<RadarTab> createState() => _RadarTabState();
}

class _RadarTabState extends State<RadarTab> with SingleTickerProviderStateMixin {
  late final AnimationController _sweep = AnimationController(
    vsync: this,
    duration: const Duration(seconds: 5),
  )..repeat();
  String? _selectedId;

  @override
  void dispose() {
    _sweep.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => AnimatedBuilder(
    animation: widget.store,
    builder: (context, _) {
      final menus = widget.store.menus;
      final selected = menus.where((menu) => menu.id == _selectedId).firstOrNull;
      return Padding(
        padding: const EdgeInsets.fromLTRB(20, 16, 20, 100),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        'Radar',
                        style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                          color: AppColors.navy,
```

```
                    fontSize: 32,
                  ),
                ),
                Text(
                  'Locales a minutos del campus',
                  style: Theme.of(context).textTheme.bodyMedium,
                ),
              ],
            ),
          ),
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
            decoration: BoxDecoration(
              color: AppColors.navy,
              borderRadius: BorderRadius.circular(100),
            ),
            child: Text(
              '${menus.length} locales',
              style: const TextStyle(
                color: Colors.white,
                fontWeight: FontWeight.w800,
                fontSize: 13,
              ),
            ),
          ),
        ),
      ],
    ),
    const SizedBox(height: 14),
    Expanded(
      child: ClipRRect(
        borderRadius: BorderRadius.circular(30),
        child: LayoutBuilder(
          builder: (context, constraints) {
            final size = constraints.biggest;
            final center = size.center(Offset.zero);
            return GestureDetector(
              behavior: HitTestBehavior.opaque,
              onTap: () => setState(() => _selectedId = null),
              child: Stack(
                children: [
                  Positioned.fill(
                    child: CustomPaint(painter: _RadarPainter(_sweep)),
                  ),
                  Positioned(
                    left: center.dx - 28,
                    top: center.dy - 24,
                    width: 56,
                    child: const _CampusMarker(),
                  ),
                  for (final menu in menus)
                    Positioned(
                      left: _pinCenter(menu, size).dx - 32,
                      top: _pinCenter(menu, size).dy - 23,
                      width: 64,
                      child: _RadarPin(
                        menu: menu,
                        selected: menu.id == _selectedId,
                        onTap: () => setState(() => _selectedId = menu.id),
                      ),
                    ),
                  const Positioned(top: 14, left: 16, right: 16, child: _Legend()),
                  Positioned(
                    left: 12,
                    right: 12,
                    bottom: 12,
                    child: AnimatedSwitcher(
                      duration: const Duration(milliseconds: 250),
                      transitionBuilder: (child, animation) => FadeTransition(
                        opacity: animation,
                        child: SlideTransition(
                          position: Tween(
                            begin: const Offset(0, 0.25),
                            end: Offset.zero,
                          ).animate(animation),
                          child: child,
                        ),
                      ),
                      child: selected == null
                          ? const _Hint(key: ValueKey('hint'))
                          : _PreviewCard(
                              key: ValueKey(selected.id),
                              menu: selected,
                              onOpen: () => widget.onOpen(selected),
                            ),
                    ),
                  ),
```

```
                        ],
                      ),
                    );
                  },
                ),
              ),
            ),
          ],
        ),
      );
    },
  );
}

class _RadarPainter extends CustomPainter {
  _RadarPainter(this.animation) : super(repaint: animation);

  final Animation<double> animation;

  static const _glow = Color(0xFFFF8A3D);

  @override
  void paint(Canvas canvas, Size size) {
    final center = size.center(Offset.zero);
    final radius = _radarRadius(size);
    canvas.drawRect(
      Offset.zero & size,
      Paint()
        ..shader = RadialGradient(
          colors: const [Color(0xFF26507A), Color(0xFF0C192B)],
        ).createShader(Rect.fromCircle(center: center, radius: size.longestSide * 0.7)),
    );

    final line = Paint()
      ..color = Colors.white.withAlpha(46)
      ..style = PaintingStyle.stroke
      ..strokeWidth = 1;
    canvas
      ..drawLine(Offset(center.dx, center.dy - radius - 14), Offset(center.dx, center.dy + radius +
14), line)
      ..drawLine(Offset(center.dx - radius - 14, center.dy), Offset(center.dx + radius + 14,
center.dy), line);
    for (final minutes in const [2, 4, radarMaxMinutes]) {
      final r = radius * radarFraction(minutes);
      canvas.drawCircle(center, r, line);
      _label(canvas, '$minutes min', Offset(center.dx + 6, center.dy - r + 3));
    }

    final angle = animation.value * 2 * math.pi;
    canvas
      ..drawCircle(
        center,
        radius,
        Paint()
          ..shader = SweepGradient(
            colors: const [Color(0x00FF8A3D), Color(0x00FF8A3D), Color(0x70FF8A3D)],
            stops: const [0, 0.7, 1],
            transform: GradientRotation(angle),
          ).createShader(Rect.fromCircle(center: center, radius: radius)),
      )
      ..drawLine(
        center,
        center + Offset(math.cos(angle), math.sin(angle)) * radius,
        Paint()
          ..color = _glow.withAlpha(190)
          ..strokeWidth = 2
          ..strokeCap = StrokeCap.round,
      );

    final pulse = (animation.value * 3) % 1;
    canvas.drawCircle(
      center,
      22 + pulse * 46,
      Paint()
        ..color = _glow.withAlpha(((1 - pulse) * 150).round())
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2,
    );
  }

  void _label(Canvas canvas, String text, Offset offset) {
    final painter = TextPainter(
      text: TextSpan(
```

```
        text: text,
        style: TextStyle(
          color: Colors.white.withAlpha(140),
          fontSize: 11,
          fontWeight: FontWeight.w700,
        ),
      ),
      textDirection: TextDirection.ltr,
    )..layout();
    painter.paint(canvas, offset);
  }

  @override
  bool shouldRepaint(covariant _RadarPainter oldDelegate) => false;
}

class _CampusMarker extends StatelessWidget {
  const _CampusMarker();

  @override
  Widget build(BuildContext context) => Column(
    mainAxisSize: MainAxisSize.min,
    children: [
      Container(
        width: 48,
        height: 48,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color: AppColors.orange,
          border: Border.all(color: Colors.white, width: 3),
          boxShadow: const [BoxShadow(color: Color(0x99FF8A3D), blurRadius: 16, spreadRadius: 2)],
        ),
        child: const Icon(Icons.school_rounded, color: Colors.white),
      ),
    ],
  );
}

class _RadarPin extends StatelessWidget {
  const _RadarPin({
    required this.menu,
    required this.selected,
    required this.onTap,
  });

  final CampusMenu menu;
  final bool selected;
  final VoidCallback onTap;

  @override
  Widget build(BuildContext context) {
    final color = availabilityColor(menu.availability);
    return Semantics(
      button: true,
      label: '${menu.restaurant}, ${menu.walkMinutes} minutos, ${menu.availability.label}',
      child: GestureDetector(
        behavior: HitTestBehavior.opaque,
        onTap: onTap,
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            AnimatedScale(
              scale: selected ? 1.2 : 1,
              duration: const Duration(milliseconds: 200),
              child: Container(
                width: 46,
                height: 46,
                decoration: BoxDecoration(
                  shape: BoxShape.circle,
                  color: color,
                  border: Border.all(color: Colors.white, width: 3),
                  boxShadow: [
                    BoxShadow(
                      color: color.withAlpha(selected ? 210 : 120),
                      blurRadius: selected ? 20 : 10,
                      spreadRadius: selected ? 4 : 1,
                    ),
                  ],
                ),
                child: Icon(categoryIcon(menu.category), color: Colors.white, size: 22),
              ),
            ),
            const SizedBox(height: 5),
            Container(
```

```
              padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(100),
              ),
              child: Text(
                'S/ ${menu.price.toStringAsFixed(0)}',
                style: AppText.display(13),
              ),
            ),
          ],
        ),
      ),
    );
  }
}

class _Legend extends StatelessWidget {
  const _Legend();

  @override
  Widget build(BuildContext context) => Wrap(
    spacing: 12,
    runSpacing: 4,
    children: [
      for (final status in MenuAvailability.values)
        Row(
          mainAxisSize: MainAxisSize.min,
          children: [
            Icon(Icons.circle, size: 9, color: availabilityColor(status)),
            const SizedBox(width: 5),
            Text(
              status.label,
              style: const TextStyle(color: Colors.white70, fontSize: 11, fontWeight:
FontWeight.w600),
            ),
          ],
        ),
    ],
  );
}

class _Hint extends StatelessWidget {
  const _Hint({super.key});

  @override
  Widget build(BuildContext context) => Container(
    padding: const EdgeInsets.symmetric(vertical: 12),
    decoration: BoxDecoration(
      color: Colors.white.withAlpha(30),
      borderRadius: BorderRadius.circular(18),
    ),
    child: const Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Icon(Icons.touch_app_rounded, color: Colors.white70, size: 20),
        SizedBox(width: 8),
        Text(
          'Toca un local',
          style: TextStyle(color: Colors.white70, fontWeight: FontWeight.w700),
        ),
      ],
    ),
  );
}

class _PreviewCard extends StatelessWidget {
  const _PreviewCard({super.key, required this.menu, required this.onOpen});

  final CampusMenu menu;
  final VoidCallback onOpen;

  @override
  Widget build(BuildContext context) => Material(
    color: Colors.white,
    borderRadius: BorderRadius.circular(22),
    clipBehavior: Clip.antiAlias,
    child: InkWell(
      onTap: onOpen,
      child: Padding(
        padding: const EdgeInsets.all(10),
        child: Row(
          children: [
            SizedBox(
```

```
              width: 68,
              height: 68,
              child: FoodBanner(
                category: menu.category,
                radius: BorderRadius.circular(16),
              ),
            ),
            const SizedBox(width: 12),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text(
                    menu.restaurant,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: AppText.display(19),
                  ),
                  const SizedBox(height: 2),
                  Text(
                    '${menu.walkMinutes} min · S/ ${menu.price.toStringAsFixed(2)}',
                    style: Theme.of(context).textTheme.bodySmall,
                  ),
                  const SizedBox(height: 6),
                  StatusBadge(status: menu.availability),
                ],
              ),
            ),
            const SizedBox(width: 8),
            Container(
              width: 42,
              height: 42,
              decoration: const BoxDecoration(
                shape: BoxShape.circle,
                color: AppColors.orange,
              ),
              child: const Icon(Icons.arrow_forward_rounded, color: Colors.white),
            ),
          ],
        ),
      ),
    ),
  );
}
```

Anexo I. lib/screens/community_tab.dart

```
import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../theme/app_theme.dart';
import '../widgets/food_art.dart';
import '../widgets/status_badge.dart';

class CommunityTab extends StatelessWidget {
  const CommunityTab({super.key, required this.store});

  final MenuStore store;

  @override
  Widget build(BuildContext context) => AnimatedBuilder(
    animation: store,
    builder: (context, _) {
      final notices = store.notices;
      return ListView(
        padding: const EdgeInsets.only(bottom: 110),
        children: [
          _Header(count: notices.length, alias: store.alias),
          const SizedBox(height: 8),
          for (var i = 0; i < notices.length; i++)
            Padding(
              padding: const EdgeInsets.fromLTRB(16, 8, 16, 8),
              child: _NoticeBubble(notice: notices[i], alignRight: i.isOdd),
            ),
          Padding(
            padding: const EdgeInsets.fromLTRB(24, 16, 24, 0),
            child: Text(
              'Avisos de ejemplo. Los nuevos duran solo esta sesión.',
              textAlign: TextAlign.center,
```

```
              style: Theme.of(context).textTheme.bodySmall,
            ),
          ),
        ],
      );
    },
  );
}

class _Header extends StatelessWidget {
  const _Header({required this.count, required this.alias});

  final int count;
  final String? alias;

  @override
  Widget build(BuildContext context) {
    final top = MediaQuery.paddingOf(context).top;
    return SizedBox(
      height: 244 + top,
      child: Stack(
        clipBehavior: Clip.none,
        children: [
          Positioned(
            top: 0,
            left: 0,
            right: 0,
            height: 206 + top,
            child: ClipPath(
              clipper: _WaveClipper(),
              child: Container(
                decoration: const BoxDecoration(
                  gradient: LinearGradient(
                    begin: Alignment.topLeft,
                    end: Alignment.bottomRight,
                    colors: [AppColors.orange, AppColors.ember],
                  ),
                ),
                child: CustomPaint(painter: _BlobPainter()),
              ),
            ),
          ),
          Positioned(
            right: -34,
            top: top + 6,
            child: Transform.rotate(
              angle: 0.35,
              child: const SizedBox(
                width: 170,
                height: 170,
                child: FoodArt(category: 'Chifa'),
              ),
            ),
          ),
          Positioned(
            left: 22,
            top: top + 28,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'Comunidad',
                  style: AppText.display(44, color: Colors.white, height: 1),
                ),
                const SizedBox(height: 8),
                Text(
                  alias == null ? 'Avisos de otros estudiantes' : 'Publicas como $alias',
                  style: TextStyle(color: Colors.white.withAlpha(220), fontSize: 15),
                ),
              ],
            ),
          ),
          Positioned(
            left: 18,
            bottom: 12,
            child: Transform.rotate(
              angle: -0.06,
              child: Container(
                padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 10),
                decoration: BoxDecoration(
                  color: AppColors.sun,
                  borderRadius: BorderRadius.circular(16),
                  boxShadow: const [
                    BoxShadow(color: Color(0x33000000), blurRadius: 12, offset: Offset(0, 4)),
                  ],
```

```
                ),
                child: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    const Icon(Icons.bolt_rounded, color: AppColors.navy),
                    const SizedBox(width: 6),
                    Text(
                      '$count avisos recientes',
                      style: AppText.display(17),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

class _WaveClipper extends CustomClipper<Path> {
  @override
  Path getClip(Size size) => Path()
    ..lineTo(0, size.height - 36)
    ..quadraticBezierTo(size.width * 0.25, size.height + 8, size.width * 0.52, size.height - 24)
    ..quadraticBezierTo(size.width * 0.80, size.height - 54, size.width, size.height - 14)
    ..lineTo(size.width, 0)
    ..close();

  @override
  bool shouldReclip(covariant CustomClipper<Path> oldClipper) => false;
}

class _BlobPainter extends CustomPainter {
  @override
  void paint(Canvas canvas, Size size) {
    final soft = Paint()..color = Colors.white.withAlpha(24);
    canvas
      ..drawCircle(Offset(size.width * 0.9, size.height * 0.1), 120, soft)
      ..drawCircle(Offset(size.width * 0.05, size.height * 0.85), 90, soft)
      ..drawCircle(Offset(size.width * 0.5, size.height * 0.05), 40, soft);
  }

  @override
  bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
}

class _NoticeBubble extends StatelessWidget {
  const _NoticeBubble({required this.notice, required this.alignRight});

  final CommunityNotice notice;
  final bool alignRight;

  @override
  Widget build(BuildContext context) {
    final status = availabilityColor(notice.availability);
    final style = categoryStyle(notice.category);
    final bubble = Expanded(
      child: Container(
        padding: const EdgeInsets.fromLTRB(18, 16, 16, 14),
        decoration: BoxDecoration(
          color: Color.alphaBlend(status.withAlpha(24), Colors.white),
          border: Border.all(color: status.withAlpha(80), width: 1.5),
          borderRadius: BorderRadius.only(
            topLeft: const Radius.circular(30),
            topRight: const Radius.circular(30),
            bottomLeft: Radius.circular(alignRight ? 30 : 6),
            bottomRight: Radius.circular(alignRight ? 6 : 30),
          ),
        ),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Row(
              children: [
                Expanded(
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        notice.restaurant,
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
```

```
                    style: AppText.display(20),
                  ),
                  Text(
                    '${notice.author} · ${_timeLabel(notice.createdAt)}',
                    style: Theme.of(context).textTheme.bodySmall,
                  ),
                ],
              ),
            ),
            const SizedBox(width: 8),
            Container(
              width: 56,
              height: 56,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                border: Border.all(color: Colors.white, width: 3),
                boxShadow: const [
                  BoxShadow(color: Color(0x33000000), blurRadius: 8, offset: Offset(0, 3)),
                ],
              ),
              child: ClipOval(child: FoodBanner(category: notice.category)),
            ),
          ],
        ),
        const SizedBox(height: 10),
        Text(
          notice.message,
          style: const TextStyle(
            color: AppColors.navy,
            fontSize: 18,
            fontWeight: FontWeight.w600,
            height: 1.25,
          ),
        ),
        const SizedBox(height: 12),
        Transform.rotate(
          angle: alignRight ? 0.04 : -0.04,
          child: StatusBadge(status: notice.availability, solid: true),
        ),
      ],
    ),
  ),
);
final avatar = Padding(
  padding: const EdgeInsets.only(top: 6),
  child: Stack(
    clipBehavior: Clip.none,
    children: [
      Container(
        width: 48,
        height: 48,
        alignment: Alignment.center,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          gradient: LinearGradient(
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
            colors: [style.top, style.bottom],
          ),
        ),
        child: Text(
          notice.author.characters.first.toUpperCase(),
          style: AppText.display(22, color: Colors.white),
        ),
      ),
      Positioned(
        right: -4,
        bottom: -4,
        child: Container(
          padding: const EdgeInsets.all(2),
          decoration: const BoxDecoration(color: Colors.white, shape: BoxShape.circle),
          child: Icon(availabilityIcon(notice.availability), size: 18, color: status),
        ),
      ),
    ],
  ),
);
return Padding(
  padding: EdgeInsets.only(left: alignRight ? 28 : 0, right: alignRight ? 0 : 28),
  child: Row(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: alignRight
        ? [bubble, const SizedBox(width: 10), avatar]
        : [avatar, const SizedBox(width: 10), bubble],
  ),
);
```

```
  }

  String _timeLabel(DateTime time) {
    final minutes = DateTime.now().difference(time).inMinutes;
    if (minutes < 1) return 'ahora';
    if (minutes < 60) return 'hace $minutes min';
    return 'hace ${minutes ~/ 60} h';
  }
}
```

Anexo J. lib/screens/menu_detail_page.dart

```
import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../theme/app_theme.dart';
import '../widgets/availability_sheet.dart';
import '../widgets/food_art.dart';
import '../widgets/status_badge.dart';

class MenuDetailPage extends StatelessWidget {
  const MenuDetailPage({super.key, required this.store, required this.menuId});

  final MenuStore store;
  final String menuId;

  static const double _expandedHeight = 280;

  @override
  Widget build(BuildContext context) => AnimatedBuilder(
    animation: store,
    builder: (context, _) {
      final menu = store.findMenu(menuId);
      final style = categoryStyle(menu.category);
      final top = MediaQuery.paddingOf(context).top;
      return Scaffold(
        body: CustomScrollView(
          slivers: [
            SliverAppBar(
              pinned: true,
              expandedHeight: _expandedHeight,
              backgroundColor: style.bottom,
              foregroundColor: Colors.white,
              automaticallyImplyLeading: false,
              leading: Padding(
                padding: const EdgeInsets.all(6),
                child: IconButton(
                  style: IconButton.styleFrom(
                    backgroundColor: Colors.white.withAlpha(70),
                    foregroundColor: Colors.white,
                  ),
                  icon: const Icon(Icons.arrow_back_rounded),
                  onPressed: () => Navigator.pop(context),
                ),
              ),
              flexibleSpace: LayoutBuilder(
                builder: (context, constraints) {
                  final collapsed = kToolbarHeight + top;
                  final t = ((constraints.maxHeight - collapsed) /
                          (_expandedHeight + top - collapsed))
                      .clamp(0.0, 1.0);
                  return Stack(
                    fit: StackFit.expand,
                    children: [
                      Hero(
                        tag: 'banner-${menu.id}',
                        child: FoodBanner(category: menu.category),
                      ),
                      Positioned(
                        top: top + 15,
                        left: 64,
                        right: 16,
                        child: Opacity(
                          opacity: 1 - t,
                          child: Text(
                            menu.restaurant,
                            maxLines: 1,
                            overflow: TextOverflow.ellipsis,
                            style: AppText.display(22, color: Colors.white),
                          ),
```

```
                    ),
                  ),
                  Positioned(
                    left: 0,
                    right: 0,
                    bottom: -1,
                    child: Container(
                      height: 28,
                      decoration: const BoxDecoration(
                        color: AppColors.cream,
                        borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
                      ),
                    ),
                  ),
                ],
              );
            },
          ),
        ),
        SliverToBoxAdapter(
          child: Padding(
            padding: const EdgeInsets.fromLTRB(20, 4, 20, 24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  menu.restaurant,
                  style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                    color: AppColors.navy,
                    fontSize: 34,
                  ),
                ),
                const SizedBox(height: 6),
                Row(
                  crossAxisAlignment: CrossAxisAlignment.center,
                  children: [
                    Text(
                      'S/ ${menu.price.toStringAsFixed(2)}',
                      style: AppText.display(36, color: AppColors.orange),
                    ),
                    const SizedBox(width: 12),
                    StatusBadge(status: menu.availability),
                  ],
                ),
                const SizedBox(height: 18),
                Row(
                  children: [
                    Expanded(
                      child: _StatTile(
                        icon: Icons.directions_walk_rounded,
                        value: '${menu.walkMinutes} min',
                        label: 'a pie',
                        color: style.top,
                      ),
                    ),
                    const SizedBox(width: 10),
                    Expanded(
                      child: _StatTile(
                        icon: Icons.place_rounded,
                        value: menu.area,
                        label: 'referencia',
                        color: style.top,
                      ),
                    ),
                    const SizedBox(width: 10),
                    Expanded(
                      child: _StatTile(
                        icon: categoryIcon(menu.category),
                        value: menu.category,
                        label: 'cocina',
                        color: style.top,
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 20),
                Card(
                  margin: EdgeInsets.zero,
                  child: Padding(
                    padding: const EdgeInsets.all(18),
                    child: Column(
                      children: [
                        _CourseRow(
                          icon: Icons.soup_kitchen_rounded,
                          title: 'Entrada',
                          items: menu.starters,
```

```
                              color: style.top,
                            ),
                            const _Divider(),
                            _CourseRow(
                              icon: Icons.restaurant_rounded,
                              title: 'Plato de fondo',
                              items: menu.mainDishes,
                              color: style.top,
                            ),
                            const _Divider(),
                            _CourseRow(
                              icon: Icons.local_drink_rounded,
                              title: 'Bebida',
                              items: [menu.drink],
                              color: style.top,
                            ),
                          ],
                        ),
                      ),
                    ),
                    const SizedBox(height: 14),
                    Row(
                      children: [
                        const Icon(Icons.info_outline_rounded, size: 16, color: Colors.black45),
                        const SizedBox(width: 6),
                        Expanded(
                          child: Text(
                            'Confirma precio y stock en el local.',
                            style: Theme.of(context).textTheme.bodySmall,
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
            ),
          ],
        ),
        bottomNavigationBar: SafeArea(
          minimum: const EdgeInsets.fromLTRB(20, 8, 20, 16),
          child: FilledButton.icon(
            onPressed: () => reportAvailability(context, store, menu),
            icon: const Icon(Icons.campaign_rounded),
            label: const Text('Avisar disponibilidad'),
          ),
        ),
      );
    },
  );
}

class _StatTile extends StatelessWidget {
  const _StatTile({
    required this.icon,
    required this.value,
    required this.label,
    required this.color,
  });

  final IconData icon;
  final String value;
  final String label;
  final Color color;

  @override
  Widget build(BuildContext context) => Container(
    padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 14),
    decoration: BoxDecoration(
      color: color.withAlpha(30),
      borderRadius: BorderRadius.circular(20),
    ),
    child: Column(
      children: [
        Icon(icon, color: color),
        const SizedBox(height: 8),
        Text(
          value,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          style: const TextStyle(
            color: AppColors.navy,
            fontWeight: FontWeight.w800,
            fontSize: 14,
          ),
```

```
        ),
        Text(label, style: Theme.of(context).textTheme.bodySmall),
      ],
    ),
  );
}

class _CourseRow extends StatelessWidget {
  const _CourseRow({
    required this.icon,
    required this.title,
    required this.items,
    required this.color,
  });

  final IconData icon;
  final String title;
  final List<String> items;
  final Color color;

  @override
  Widget build(BuildContext context) => Row(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Container(
        width: 46,
        height: 46,
        decoration: BoxDecoration(
          color: color.withAlpha(32),
          borderRadius: BorderRadius.circular(15),
        ),
        child: Icon(icon, color: color),
      ),
      const SizedBox(width: 14),
      Expanded(
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(title.toUpperCase(), style: Theme.of(context).textTheme.labelSmall),
            const SizedBox(height: 4),
            for (final item in items)
              Padding(
                padding: const EdgeInsets.only(bottom: 2),
                child: Text(
                  item,
                  style: const TextStyle(
                    fontSize: 17,
                    fontWeight: FontWeight.w700,
                    color: AppColors.navy,
                  ),
                ),
              ),
          ],
        ),
      ),
    ],
  );
}

class _Divider extends StatelessWidget {
  const _Divider();

  @override
  Widget build(BuildContext context) => const Padding(
    padding: EdgeInsets.symmetric(vertical: 14),
    child: Divider(height: 1),
  );
}
```

Anexo K. lib/screens/publish_menu_page.dart

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../theme/app_theme.dart';
import '../widgets/food_art.dart';
import '../widgets/status_badge.dart';
```

```
class PublishMenuPage extends StatefulWidget {
  const PublishMenuPage({super.key, required this.store});

  final MenuStore store;

  @override
  State<PublishMenuPage> createState() => _PublishMenuPageState();
}

class _PublishMenuPageState extends State<PublishMenuPage> {
  static const _categories = ['Criollo', 'Vegetariano', 'Chifa', 'Otro'];

  final _formKey = GlobalKey<FormState>();
  final _restaurant = TextEditingController();
  final _price = TextEditingController();
  final _walk = TextEditingController();
  final _area = TextEditingController();
  final _starter = TextEditingController();
  final _dish = TextEditingController();
  final _drink = TextEditingController();
  String _category = 'Criollo';
  MenuAvailability _availability = MenuAvailability.available;

  @override
  void dispose() {
    for (final controller in [
      _restaurant, _price, _walk, _area, _starter, _dish, _drink,
    ]) {
      controller.dispose();
    }
    super.dispose();
  }

  String? _required(String? value) => value == null || value.trim().isEmpty
      ? 'Completa este campo.'
      : null;

  /// Elige el ángulo del radar más alejado de los pines que ya existen, para
  /// que un menú nuevo no se encime con otro.
  double _freeAngle(int walk) {
    final mine = radarFraction(walk);
    final taken = [
      for (final menu in widget.store.menus)
        Offset.fromDirection(
          math.atan2(menu.positionY - 0.5, menu.positionX - 0.5),
          radarFraction(menu.walkMinutes),
        ),
    ];
    var best = 0.0;
    var bestGap = -1.0;
    for (var k = 0; k < 24; k++) {
      final angle = k * math.pi / 12;
      final point = Offset.fromDirection(angle, mine);
      final gap = taken.isEmpty
          ? 1.0
          : taken.map((other) => (other - point).distance).reduce(math.min);
      if (gap > bestGap) {
        best = angle;
        bestGap = gap;
      }
    }
    return best;
  }

  void _submit() {
    if (!_formKey.currentState!.validate()) return;
    final price = double.parse(_price.text.trim().replaceAll(',', '.'));
    final walk = int.parse(_walk.text.trim());
    final id = DateTime.now().microsecondsSinceEpoch.toString();
    final angle = _freeAngle(walk);
    widget.store.addMenu(
      CampusMenu(
        id: id,
        restaurant: _restaurant.text.trim(),
        price: price,
        walkMinutes: walk,
        area: _area.text.trim(),
        category: _category,
        starters: [_starter.text.trim()],
        mainDishes: [_dish.text.trim()],
        drink: _drink.text.trim(),
        availability: _availability,
```

```
      positionX: 0.5 + 0.35 * math.cos(angle),
      positionY: 0.5 + 0.35 * math.sin(angle),
    ),
  );
  Navigator.pop(context, id);
}

Widget _field({
  required TextEditingController controller,
  required String label,
  required IconData icon,
  String? hint,
  String? prefixText,
  String? suffixText,
  TextInputType? keyboardType,
  String? Function(String?)? validator,
  bool last = false,
}) => TextFormField(
  controller: controller,
  keyboardType: keyboardType,
  autovalidateMode: AutovalidateMode.onUserInteraction,
  textInputAction: last ? TextInputAction.done : TextInputAction.next,
  textCapitalization: keyboardType == null
      ? TextCapitalization.sentences
      : TextCapitalization.none,
  decoration: InputDecoration(
    labelText: label,
    hintText: hint,
    prefixIcon: Icon(icon),
    prefixText: prefixText,
    suffixText: suffixText,
  ),
  validator: validator ?? _required,
);

@override
Widget build(BuildContext context) => Scaffold(
  appBar: AppBar(title: const Text('Publicar menú')),
  body: SafeArea(
    child: Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.fromLTRB(20, 4, 20, 24),
        children: [
          _Preview(
            category: _category,
            restaurant: _restaurant,
            price: _price,
          ),
          const SizedBox(height: 14),
          SizedBox(
            height: 44,
            child: ListView.separated(
              scrollDirection: Axis.horizontal,
              itemCount: _categories.length,
              separatorBuilder: (context, index) => const SizedBox(width: 8),
              itemBuilder: (context, index) {
                final category = _categories[index];
                return ChoiceChip(
                  avatar: Icon(categoryIcon(category), size: 18),
                  label: Text(category),
                  showCheckmark: false,
                  selected: _category == category,
                  onSelected: (_) => setState(() => _category = category),
                );
              },
            ),
          ),
          const SizedBox(height: 24),
          _Section(
            number: 1,
            title: 'El local',
            children: [
              _field(
                controller: _restaurant,
                label: 'Nombre del local',
                hint: 'Ej. La Tía Rosa',
                icon: Icons.storefront_rounded,
              ),
              _field(
                controller: _area,
                label: 'Referencia cercana',
                hint: 'Ej. Puerta 3',
                icon: Icons.place_outlined,
              ),
              Row(
```

```
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Expanded(
                    child: _field(
                      controller: _walk,
                      label: 'A pie',
                      icon: Icons.directions_walk_rounded,
                      suffixText: 'min',
                      keyboardType: TextInputType.number,
                      validator: (value) {
                        final parsed = int.tryParse((value ?? '').trim());
                        return parsed == null || parsed < 1 || parsed > 60
                            ? '1 a 60.'
                            : null;
                      },
                    ),
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    child: _field(
                      controller: _price,
                      label: 'Precio',
                      icon: Icons.payments_outlined,
                      prefixText: 'S/ ',
                      keyboardType: const TextInputType.numberWithOptions(decimal: true),
                      validator: (value) {
                        final parsed = double.tryParse(
                          (value ?? '').trim().replaceAll(',', '.'),
                        );
                        return parsed == null || parsed <= 0 ? 'Precio inválido.' : null;
                      },
                    ),
                  ),
                ],
              ),
            ],
          ),
          const SizedBox(height: 22),
          _Section(
            number: 2,
            title: 'El menú de hoy',
            children: [
              _field(
                controller: _starter,
                label: 'Entrada',
                hint: 'Ej. Sopa del día',
                icon: Icons.soup_kitchen_rounded,
              ),
              _field(
                controller: _dish,
                label: 'Plato de fondo',
                hint: 'Ej. Lomo saltado',
                icon: Icons.restaurant_rounded,
              ),
              _field(
                controller: _drink,
                label: 'Bebida',
                hint: 'Ej. Chicha morada',
                icon: Icons.local_drink_rounded,
                last: true,
              ),
            ],
          ),
          const SizedBox(height: 22),
          _Section(
            number: 3,
            title: '¿Cuánto queda?',
            children: [
              Row(
                children: [
                  Expanded(
                    child: SegmentedButton<MenuAvailability>(
                      showSelectedIcon: false,
                      style: SegmentedButton.styleFrom(
                        textStyle: const TextStyle(fontSize: 12, fontWeight: FontWeight.w600),
                      ),
                      segments: [
                        for (final (status, label) in const [
                          (MenuAvailability.available, 'Hay'),
                          (MenuAvailability.limited, 'Pocas'),
                          (MenuAvailability.soldOut, 'Agotado'),
                        ])
                          ButtonSegment(
                            value: status,
                            icon: Icon(availabilityIcon(status), size: 18),
                            label: Text(label),
```

```
                          ),
                        ],
                        selected: {_availability},
                        onSelectionChanged: (values) =>
                            setState(() => _availability = values.first),
                      ),
                    ),
                  ],
                ),
              ],
            ),
            const SizedBox(height: 22),
            FilledButton.icon(
              onPressed: _submit,
              icon: const Icon(Icons.publish_rounded),
              label: const Text('Publicar menú'),
            ),
            const SizedBox(height: 10),
            Text(
              'Se guarda solo mientras la app está abierta.',
              textAlign: TextAlign.center,
              style: Theme.of(context).textTheme.bodySmall,
            ),
          ],
        ),
      ),
    ),
  );
}

/// Vista previa en vivo del banner que verán los demás estudiantes.
class _Preview extends StatelessWidget {
  const _Preview({
    required this.category,
    required this.restaurant,
    required this.price,
  });

  final String category;
  final TextEditingController restaurant;
  final TextEditingController price;

  @override
  Widget build(BuildContext context) => SizedBox(
    height: 150,
    child: FoodBanner(
      category: category,
      radius: BorderRadius.circular(28),
      child: ListenableBuilder(
        listenable: Listenable.merge([restaurant, price]),
        builder: (context, _) {
          final name = restaurant.text.trim();
          final value = price.text.trim();
          return Padding(
            padding: const EdgeInsets.fromLTRB(18, 14, 18, 14),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.end,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'Así se verá',
                  style: TextStyle(
                    color: Colors.white.withAlpha(200),
                    fontSize: 12,
                    fontWeight: FontWeight.w600,
                  ),
                ),
                Text(
                  value.isEmpty ? 'S/ --' : 'S/ $value',
                  style: AppText.display(34, color: Colors.white),
                ),
                SizedBox(
                  width: 190,
                  child: Text(
                    name.isEmpty ? 'Tu local' : name,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: AppText.display(18, color: Colors.white),
                  ),
                ),
              ],
            ),
          );
        },
      ),
```

```
    ),
  );
}

class _Section extends StatelessWidget {
  const _Section({
    required this.number,
    required this.title,
    required this.children,
  });

  final int number;
  final String title;
  final List<Widget> children;

  @override
  Widget build(BuildContext context) => Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Row(
        children: [
          Container(
            width: 30,
            height: 30,
            alignment: Alignment.center,
            decoration: const BoxDecoration(
              color: AppColors.orange,
              shape: BoxShape.circle,
            ),
            child: Text('$number', style: AppText.display(16, color: Colors.white)),
          ),
          const SizedBox(width: 10),
          Text(title, style: AppText.display(22)),
        ],
      ),
      const SizedBox(height: 14),
      for (var i = 0; i < children.length; i++) ...[
        if (i > 0) const SizedBox(height: 12),
        children[i],
      ],
    ],
  );
}
```

Anexo L. lib/widgets/food_art.dart

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

/// Colores asociados a cada tipo de cocina.
class CategoryStyle {
  const CategoryStyle({
    required this.top,
    required this.bottom,
    required this.accent,
  });

  final Color top;
  final Color bottom;
  final Color accent;
}

CategoryStyle categoryStyle(String category) => switch (category) {
  'Vegetariano' => const CategoryStyle(
    top: Color(0xFF34B386),
    bottom: Color(0xFF0B6B52),
    accent: Color(0xFFC6F5DC),
  ),
  'Chifa' => const CategoryStyle(
    top: Color(0xFFEE5A45),
    bottom: Color(0xFF9E1F2B),
    accent: Color(0xFFFFD98A),
  ),
  'Criollo' => const CategoryStyle(
    top: Color(0xFFF8A02E),
    bottom: Color(0xFFD4510B),
    accent: Color(0xFFFFE6B8),
  ),
  _ => const CategoryStyle(
```

```
    top: Color(0xFF6A7BA2),
    bottom: Color(0xFF14233A),
    accent: Color(0xFFDDE5F7),
  ),
};

IconData categoryIcon(String category) => switch (category) {
  'Vegetariano' => Icons.eco_rounded,
  'Chifa' => Icons.ramen_dining_rounded,
  'Criollo' => Icons.restaurant_rounded,
  _ => Icons.lunch_dining_rounded,
};

/// Fondo con degradado y la ilustración del plato asomando por una esquina.
/// Si [height] es nulo, ocupa el alto que le entregue el padre.
class FoodBanner extends StatelessWidget {
  const FoodBanner({
    super.key,
    required this.category,
    this.height,
    this.radius = BorderRadius.zero,
    this.child,
  });

  final String category;
  final double? height;
  final BorderRadius radius;
  final Widget? child;

  @override
  Widget build(BuildContext context) {
    final style = categoryStyle(category);
    return ClipRRect(
      borderRadius: radius,
      child: SizedBox(
        height: height,
        width: double.infinity,
        child: Stack(
          fit: StackFit.expand,
          children: [
            DecoratedBox(
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  begin: Alignment.topLeft,
                  end: Alignment.bottomRight,
                  colors: [style.top, style.bottom],
                ),
              ),
            ),
            CustomPaint(painter: _BackdropPainter(style.accent)),
            LayoutBuilder(
              builder: (context, constraints) {
                final side =
                    math.min(constraints.maxHeight, constraints.maxWidth) * 1.1;
                return Stack(
                  children: [
                    Positioned(
                      right: -side * 0.06,
                      bottom: -side * 0.14,
                      width: side,
                      height: side,
                      child: FoodArt(category: category),
                    ),
                  ],
                );
              },
            ),
            if (child != null) child!,
          ],
        ),
      ),
    );
  }
}

/// Ilustración vectorial del plato típico de cada cocina.
class FoodArt extends StatelessWidget {
  const FoodArt({super.key, required this.category});

  final String category;

  @override
  Widget build(BuildContext context) => RepaintBoundary(
    child: CustomPaint(painter: _FoodPainter(category)),
```

```
  );
}

class _BackdropPainter extends CustomPainter {
  _BackdropPainter(this.accent);

  final Color accent;

  @override
  void paint(Canvas canvas, Size size) {
    canvas.drawCircle(
      Offset(size.width * 0.78, size.height * 0.25),
      size.height * 0.62,
      Paint()..color = accent.withAlpha(38),
    );
    canvas.drawCircle(
      Offset(size.width * 0.04, size.height),
      size.height * 0.55,
      Paint()..color = Colors.white.withAlpha(22),
    );
    final dot = Paint()..color = Colors.white.withAlpha(70);
    for (var i = 0; i < 5; i++) {
      for (var j = 0; j < 3; j++) {
        canvas.drawCircle(Offset(18 + i * 12.0, 18 + j * 12.0), 1.7, dot);
      }
    }
  }

  @override
  bool shouldRepaint(covariant _BackdropPainter oldDelegate) =>
      oldDelegate.accent != accent;
}

Paint _fill(Color color) => Paint()..color = color;

Paint _stroke(Color color, double width) => Paint()
  ..color = color
  ..style = PaintingStyle.stroke
  ..strokeWidth = width
  ..strokeCap = StrokeCap.round;

void _bar(
  Canvas canvas,
  Offset center,
  double width,
  double height,
  double angle,
  Color color,
) {
  canvas
    ..save()
    ..translate(center.dx, center.dy)
    ..rotate(angle)
    ..drawRRect(
      RRect.fromRectAndRadius(
        Rect.fromCenter(center: Offset.zero, width: width, height: height),
        Radius.circular(math.min(width, height) * 0.45),
      ),
      _fill(color),
    )
    ..restore();
}

class _FoodPainter extends CustomPainter {
  _FoodPainter(this.category);

  final String category;

  @override
  void paint(Canvas canvas, Size size) {
    canvas
      ..save()
      ..scale(size.width, size.height);
    switch (category) {
      case 'Criollo':
        _criollo(canvas);
      case 'Vegetariano':
        _vegetariano(canvas);
      case 'Chifa':
        _chifa(canvas);
      default:
        _cloche(canvas);
```

```
  }
  canvas.restore();
}

/// Lomo saltado visto desde arriba: arroz, papas fritas y tiras de carne.
void _criollo(Canvas c) {
  c
    ..drawCircle(const Offset(0.52, 0.58), 0.40, _fill(const Color(0x33000000)))
    ..drawCircle(const Offset(0.5, 0.52), 0.40, _fill(Colors.white))
    ..drawCircle(const Offset(0.5, 0.52), 0.31, _fill(const Color(0xFFF1EBE0)))
    ..drawOval(
      Rect.fromCenter(center: const Offset(0.37, 0.60), width: 0.30, height: 0.24),
      _fill(const Color(0xFFFFF7E0)),
    );
  final grain = _stroke(const Color(0xFFE6D5A8), 0.008);
  for (var i = 0; i < 10; i++) {
    final o = Offset(
      0.37 + 0.10 * math.cos(i * 1.7) * ((i % 3) + 1) / 3,
      0.60 + 0.07 * math.sin(i * 2.3) * ((i % 3) + 1) / 3,
    );
    c.drawLine(o, o.translate(0.022, 0.010), grain);
  }
  const fries = [
    (0.60, 0.66, 0.35),
    (0.65, 0.61, -0.20),
    (0.70, 0.67, 0.80),
    (0.63, 0.72, -0.45),
    (0.69, 0.58, 0.10),
    (0.58, 0.60, 1.20),
  ];
  for (final (x, y, a) in fries) {
    _bar(c, Offset(x, y), 0.15, 0.036, a, const Color(0xFFF4B73A));
  }
  const meat = [
    (0.46, 0.40, 0.4),
    (0.53, 0.37, -0.5),
    (0.60, 0.42, 0.9),
    (0.50, 0.46, -0.2),
    (0.40, 0.45, 1.1),
  ];
  for (final (x, y, a) in meat) {
    _bar(c, Offset(x, y), 0.12, 0.045, a, const Color(0xFF7A3F22));
  }
  final onion = _stroke(const Color(0xFFC77BBE), 0.018);
  c
    ..drawArc(
      Rect.fromCenter(center: const Offset(0.45, 0.35), width: 0.10, height: 0.07),
      0.2,
      2.6,
      false,
      onion,
    )
    ..drawArc(
      Rect.fromCenter(center: const Offset(0.58, 0.47), width: 0.10, height: 0.07),
      3.4,
      2.6,
      false,
      onion,
    )
    ..drawCircle(const Offset(0.40, 0.36), 0.032, _fill(const Color(0xFFE34B32)))
    ..drawCircle(const Offset(0.55, 0.50), 0.028, _fill(const Color(0xFFE34B32)))
    ..drawCircle(const Offset(0.65, 0.40), 0.026, _fill(const Color(0xFFE34B32)));
  for (final (x, y, a) in const [(0.47, 0.30, -0.6), (0.53, 0.29, 0.5)]) {
    _bar(c, Offset(x, y), 0.07, 0.035, a, const Color(0xFF3E9A4F));
  }
}

void _bowl(Canvas c, Color wall, Color band, Color inner) {
  final rim = Rect.fromCenter(center: const Offset(0.5, 0.50), width: 0.76, height: 0.20);
  c.drawOval(rim, _fill(inner));
  final body = Path()
    ..moveTo(0.12, 0.50)
    ..arcTo(rim, math.pi, -math.pi, false)
    ..arcTo(
      Rect.fromCenter(center: const Offset(0.5, 0.50), width: 0.76, height: 0.76),
      0,
      math.pi,
      false,
    )
    ..close();
  c
    ..save()
    ..clipPath(body)
    ..drawRect(const Rect.fromLTWH(0, 0.50, 1, 0.5), _fill(wall))
    ..drawRect(const Rect.fromLTWH(0, 0.70, 1, 0.05), _fill(band))
    ..restore()
```

```
    ..drawPath(body, _stroke(Colors.black.withAlpha(24), 0.008));
}

void _bowlFront(Canvas c) {
  // Labio frontal para que el contenido parezca estar dentro del bowl.
  c.drawArc(
    Rect.fromCenter(center: const Offset(0.5, 0.50), width: 0.76, height: 0.20),
    0,
    math.pi,
    false,
    _stroke(Colors.white.withAlpha(200), 0.012),
  );
}

void _vegetariano(Canvas c) {
  _bowl(c, Colors.white, const Color(0xFF34B386), const Color(0xFFD9EDE3));
  const leaves = [
    (0.30, 0.44, 0.16, 0.10, -0.5, Color(0xFF58C46E)),
    (0.42, 0.38, 0.18, 0.10, 0.3, Color(0xFF3FA95A)),
    (0.56, 0.36, 0.18, 0.10, -0.2, Color(0xFF6ED084)),
    (0.68, 0.43, 0.16, 0.10, 0.6, Color(0xFF3FA95A)),
    (0.50, 0.46, 0.20, 0.10, 0.0, Color(0xFF58C46E)),
  ];
  for (final (x, y, w, h, a, color) in leaves) {
    c
      ..save()
      ..translate(x, y)
      ..rotate(a)
      ..drawOval(Rect.fromCenter(center: Offset.zero, width: w, height: h), _fill(color))
      ..restore();
  }
  c
    ..drawCircle(const Offset(0.36, 0.40), 0.065, _fill(const Color(0xFFE5483A)))
    ..drawCircle(const Offset(0.34, 0.38), 0.016, _fill(Colors.white.withAlpha(150)))
    ..drawCircle(const Offset(0.62, 0.39), 0.058, _fill(const Color(0xFFE5483A)))
    ..drawCircle(const Offset(0.60, 0.37), 0.014, _fill(Colors.white.withAlpha(150)))
    ..drawCircle(const Offset(0.50, 0.40), 0.040, _fill(const Color(0xFFF59A2B)))
    ..drawCircle(const Offset(0.50, 0.40), 0.016, _fill(const Color(0xFFFFD08A)))
    ..drawCircle(const Offset(0.70, 0.47), 0.032, _fill(const Color(0xFFF59A2B)));
  _bowlFront(c);
  // Hojas decorativas sobre el fondo.
  c
    ..drawCircle(const Offset(0.86, 0.14), 0.035, _fill(Colors.white.withAlpha(120)))
    ..drawCircle(const Offset(0.12, 0.22), 0.022, _fill(Colors.white.withAlpha(120)));
}

void _chifa(Canvas c) {
  _bowl(c, const Color(0xFFFFF4E0), const Color(0xFFE5483A), const Color(0xFF7B1B24));
  final noodle = _stroke(const Color(0xFFF7D26A), 0.024);
  for (var k = 0; k < 7; k++) {
    final x = 0.26 + 0.08 * k;
    c.drawPath(
      Path()
        ..moveTo(x, 0.52)
        ..cubicTo(x - 0.05, 0.40, x + 0.07, 0.36, x + 0.02, 0.26)
        ..cubicTo(x - 0.02, 0.20, x + 0.05, 0.18, x + 0.04, 0.16),
      noodle,
    );
  }
  for (final (x, y) in const [(0.40, 0.40), (0.56, 0.34), (0.66, 0.44), (0.48, 0.47)]) {
    c
      ..drawCircle(Offset(x, y), 0.026, _fill(const Color(0xFF7BCB4A)))
      ..drawCircle(Offset(x, y), 0.012, _fill(const Color(0xFFD7F3B5)));
  }
  _bowlFront(c);
  final stick = _stroke(const Color(0xFFD9A15C), 0.020);
  c
    ..drawLine(const Offset(0.80, 0.06), const Offset(0.52, 0.50), stick)
    ..drawLine(const Offset(0.90, 0.12), const Offset(0.60, 0.52), stick);
  final steam = _stroke(Colors.white.withAlpha(130), 0.016);
  for (final x in const [0.30, 0.40]) {
    c.drawPath(
      Path()
        ..moveTo(x, 0.20)
        ..cubicTo(x - 0.05, 0.14, x + 0.05, 0.09, x, 0.03),
      steam,
    );
  }
}

void _cloche(Canvas c) {
  c
    ..drawOval(
      Rect.fromCenter(center: const Offset(0.5, 0.74), width: 0.84, height: 0.12),
      _fill(const Color(0x33000000)),
```

```
      )
      ..drawRRect(
        RRect.fromRectAndRadius(
          const Rect.fromLTWH(0.10, 0.66, 0.80, 0.07),
          const Radius.circular(0.035),
        ),
        _fill(const Color(0xFFE9EEF8)),
      )
      ..drawArc(
        Rect.fromCenter(center: const Offset(0.5, 0.66), width: 0.70, height: 0.70),
        math.pi,
        math.pi,
        true,
        _fill(const Color(0xFFF5F8FF)),
      )
      ..drawArc(
        Rect.fromCenter(center: const Offset(0.5, 0.66), width: 0.54, height: 0.54),
        math.pi * 1.1,
        math.pi * 0.5,
        false,
        _stroke(Colors.white, 0.03),
      )
      ..drawCircle(const Offset(0.5, 0.28), 0.045, _fill(const Color(0xFFE9EEF8)));
    for (final (x, y, r) in const [(0.86, 0.22, 0.05), (0.14, 0.40, 0.035), (0.80, 0.40, 0.025)]) {
      c
        ..drawLine(Offset(x - r, y), Offset(x + r, y), _stroke(Colors.white, 0.012))
        ..drawLine(Offset(x, y - r), Offset(x, y + r), _stroke(Colors.white, 0.012));
    }
  }

  @override
  bool shouldRepaint(covariant _FoodPainter oldDelegate) =>
      oldDelegate.category != category;
}
```

Anexo M. lib/widgets/menu_card.dart

```
import 'package:flutter/material.dart';

import '../models/menu.dart';
import '../theme/app_theme.dart';
import 'food_art.dart';
import 'status_badge.dart';

class MenuCard extends StatelessWidget {
  const MenuCard({
    super.key,
    required this.menu,
    required this.onOpen,
    required this.onReport,
  });

  final CampusMenu menu;
  final VoidCallback onOpen;
  final VoidCallback onReport;

  @override
  Widget build(BuildContext context) {
    final style = categoryStyle(menu.category);
    final soldOut = menu.availability == MenuAvailability.soldOut;
    return Card(
      clipBehavior: Clip.antiAlias,
      margin: EdgeInsets.zero,
      child: InkWell(
        onTap: onOpen,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Stack(
              children: [
                Hero(
                  tag: 'banner-${menu.id}',
                  child: FoodBanner(category: menu.category, height: 156),
                ),
                if (soldOut) const Positioned.fill(child: _SoldOutVeil()),
                Positioned(
                  top: 12,
                  left: 12,
                  child: StatusBadge(status: menu.availability, solid: true),
                ),
                Positioned(
```

```
                left: 16,
                bottom: 12,
                child: Text(
                  'S/ ${menu.price.toStringAsFixed(2)}',
                  style: AppText.display(
                    36,
                    color: Colors.white,
                    shadows: const [Shadow(color: Color(0x55000000), blurRadius: 8)],
                  ),
                ),
              ),
            ],
          ),
          Padding(
            padding: const EdgeInsets.fromLTRB(16, 14, 8, 14),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Row(
                  children: [
                    Expanded(
                      child: Text(
                        menu.restaurant,
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
                        style: Theme.of(context).textTheme.titleLarge?.copyWith(
                          color: AppColors.navy,
                          fontSize: 23,
                        ),
                      ),
                    ),
                    IconButton.filledTonal(
                      tooltip: 'Avisar stock',
                      onPressed: onReport,
                      icon: const Icon(Icons.campaign_rounded),
                    ),
                  ],
                ),
                Padding(
                  padding: const EdgeInsets.only(right: 8),
                  child: Wrap(
                    spacing: 14,
                    runSpacing: 6,
                    children: [
                      _Meta(icon: Icons.directions_walk_rounded, text: '${menu.walkMinutes} min'),
                      _Meta(icon: Icons.place_outlined, text: menu.area),
                      _Meta(icon: categoryIcon(menu.category), text: menu.category),
                    ],
                  ),
                ),
                const SizedBox(height: 12),
                Padding(
                  padding: const EdgeInsets.only(right: 8),
                  child: Wrap(
                    spacing: 8,
                    runSpacing: 8,
                    children: [
                      for (final dish in menu.mainDishes)
                        Container(
                          padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
                          decoration: BoxDecoration(
                            color: style.top.withAlpha(30),
                            borderRadius: BorderRadius.circular(100),
                          ),
                          child: Text(
                            dish,
                            style: TextStyle(
                              color: style.bottom,
                              fontWeight: FontWeight.w700,
                              fontSize: 13,
                            ),
                          ),
                        ),
                    ],
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  );
  }
}
```

```
class _Meta extends StatelessWidget {
  const _Meta({required this.icon, required this.text});

  final IconData icon;
  final String text;

  @override
  Widget build(BuildContext context) => Row(
    mainAxisSize: MainAxisSize.min,
    children: [
      Icon(icon, size: 16, color: Colors.black54),
      const SizedBox(width: 4),
      Text(text, style: Theme.of(context).textTheme.bodySmall),
    ],
  );
}

class _SoldOutVeil extends StatelessWidget {
  const _SoldOutVeil();

  @override
  Widget build(BuildContext context) => Container(
    color: Colors.black.withAlpha(110),
    alignment: Alignment.center,
    child: Transform.rotate(
      angle: -0.2,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 4),
        decoration: BoxDecoration(
          border: Border.all(color: Colors.white, width: 3),
          borderRadius: BorderRadius.circular(10),
        ),
        child: Text(
          'AGOTADO',
          style: AppText.display(26, color: Colors.white, letterSpacing: 3),
        ),
      ),
    ),
  );
}
```

Anexo N. lib/widgets/status_badge.dart

```
import 'package:flutter/material.dart';

import '../models/menu.dart';
import '../theme/app_theme.dart';

Color availabilityColor(MenuAvailability status) => switch (status) {
  MenuAvailability.available => AppColors.green,
  MenuAvailability.limited => AppColors.amber,
  MenuAvailability.soldOut => AppColors.red,
};

IconData availabilityIcon(MenuAvailability status) => switch (status) {
  MenuAvailability.available => Icons.check_circle_outline,
  MenuAvailability.limited => Icons.warning_amber_rounded,
  MenuAvailability.soldOut => Icons.cancel_outlined,
};

class StatusBadge extends StatelessWidget {
  const StatusBadge({super.key, required this.status, this.solid = false});

  final MenuAvailability status;

  /// Fondo blanco con sombra, para colocarlo sobre imágenes.
  final bool solid;

  @override
  Widget build(BuildContext context) {
    final color = availabilityColor(status);
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 6),
      decoration: BoxDecoration(
        color: solid ? Colors.white : color.withAlpha(22),
        borderRadius: BorderRadius.circular(100),
        boxShadow: solid
            ? const [BoxShadow(color: Color(0x33000000), blurRadius: 8, offset: Offset(0, 2))]
```

```
            : null,
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          Icon(availabilityIcon(status), size: 15, color: color),
          const SizedBox(width: 5),
          Text(
            status.label,
            style: TextStyle(
              color: color,
              fontWeight: FontWeight.w700,
              fontSize: 12,
            ),
          ),
        ],
      ),
    );
  }
}
```

Anexo O. lib/widgets/availability_sheet.dart

```
import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import 'status_badge.dart';

Future<void> reportAvailability(
  BuildContext context,
  MenuStore store,
  CampusMenu menu,
) async {
  final selected = await showModalBottomSheet<MenuAvailability>(
    context: context,
    isScrollControlled: true,
    showDragHandle: true,
    builder: (sheetContext) => SafeArea(
      child: Padding(
        padding: const EdgeInsets.fromLTRB(20, 4, 20, 20),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              'Disponibilidad de ${menu.restaurant}',
              style: Theme.of(sheetContext).textTheme.titleLarge,
            ),
            const SizedBox(height: 8),
            const Text('Selecciona el estado que observaste en el local.'),
            const SizedBox(height: 16),
            for (final status in MenuAvailability.values)
              ListTile(
                contentPadding: EdgeInsets.zero,
                leading: Icon(availabilityIcon(status), color: availabilityColor(status)),
                title: Text(status.label),
                trailing: menu.availability == status
                    ? const Icon(Icons.check_rounded)
                    : null,
                onTap: () => Navigator.pop(sheetContext, status),
              ),
            const SizedBox(height: 6),
            Text(
              'Prototipo: el cambio se guarda solo mientras la app está abierta.',
              style: Theme.of(sheetContext).textTheme.bodySmall,
            ),
          ],
        ),
      ),
    ),
  );
  if (selected == null || !context.mounted) return;
  store.updateAvailability(menu.id, selected);
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(content: Text('Disponibilidad actualizada en esta sesión.')),
  );
}
```

Anexo P. lib/widgets/notice_composer.dart

```
import 'package:flutter/material.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../theme/app_theme.dart';
import 'food_art.dart';
import 'status_badge.dart';

/// Abre la hoja para publicar un aviso en Comunidad. Devuelve `true` si se
/// publicó.
Future<bool> showNoticeComposer(BuildContext context, MenuStore store) async {
  final published = await showModalBottomSheet<bool>(
    context: context,
    isScrollControlled: true,
    showDragHandle: true,
    builder: (_) => _NoticeComposer(store: store),
  );
  return published ?? false;
}

class _NoticeComposer extends StatefulWidget {
  const _NoticeComposer({required this.store});

  final MenuStore store;

  @override
  State<_NoticeComposer> createState() => _NoticeComposerState();
}

class _NoticeComposerState extends State<_NoticeComposer> {
  final _formKey = GlobalKey<FormState>();
  final _alias = TextEditingController();
  final _comment = TextEditingController();
  String? _menuId;
  MenuAvailability _status = MenuAvailability.limited;
  bool _missingMenu = false;

  bool get _needsAlias => widget.store.alias == null;

  @override
  void dispose() {
    _alias.dispose();
    _comment.dispose();
    super.dispose();
  }

  void _submit() {
    final valid = _formKey.currentState!.validate();
    setState(() => _missingMenu = _menuId == null);
    if (!valid || _menuId == null) return;
    if (_needsAlias) widget.store.setAlias(_alias.text);
    widget.store.updateAvailability(_menuId!, _status, comment: _comment.text);
    Navigator.pop(context, true);
  }

  @override
  Widget build(BuildContext context) {
    final menus = widget.store.menus;
    return Padding(
      padding: EdgeInsets.only(bottom: MediaQuery.viewInsetsOf(context).bottom),
      child: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.fromLTRB(20, 0, 20, 16),
          child: Form(
            key: _formKey,
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text('Nuevo aviso', style: AppText.display(28)),
                const SizedBox(height: 4),
                Text(
                  'Cuéntale a otros qué viste en el local.',
                  style: Theme.of(context).textTheme.bodyMedium,
                ),
                const SizedBox(height: 18),
                if (_needsAlias) ...[
                  TextFormField(
                    controller: _alias,
                    autovalidateMode: AutovalidateMode.onUserInteraction,
                    textCapitalization: TextCapitalization.words,
```

```
    maxLength: 16,
    decoration: const InputDecoration(
      labelText: 'Tu apodo',
      hintText: 'Ej. Valeria R.',
      prefixIcon: Icon(Icons.badge_outlined),
      helperText: 'Sin cuenta ni correo: solo se muestra el apodo.',
    ),
    validator: (value) => (value ?? '').trim().length < 2
        ? 'Usa al menos 2 letras.'
        : null,
  ),
  const SizedBox(height: 12),
],
const _Label(icon: Icons.storefront_rounded, text: '¿Dónde?'),
const SizedBox(height: 8),
Wrap(
  spacing: 8,
  runSpacing: 8,
  children: [
    for (final menu in menus)
      ChoiceChip(
        avatar: Icon(categoryIcon(menu.category), size: 18),
        label: Text(menu.restaurant),
        showCheckmark: false,
        selected: _menuId == menu.id,
        onSelected: (_) => setState(() {
          _menuId = menu.id;
          _missingMenu = false;
        }),
      ),
  ],
),
if (_missingMenu)
  const Padding(
    padding: EdgeInsets.only(top: 6),
    child: Text(
      'Elige un local.',
      style: TextStyle(color: AppColors.red, fontSize: 12),
    ),
  ),
const SizedBox(height: 18),
const _Label(icon: Icons.restaurant_rounded, text: '¿Cómo está?'),
const SizedBox(height: 8),
Row(
  children: [
    Expanded(
      child: SegmentedButton<MenuAvailability>(
        showSelectedIcon: false,
        style: SegmentedButton.styleFrom(
          textStyle: const TextStyle(fontSize: 12, fontWeight: FontWeight.w600),
        ),
        segments: [
          for (final (status, label) in const [
            (MenuAvailability.available, 'Hay'),
            (MenuAvailability.limited, 'Pocas'),
            (MenuAvailability.soldOut, 'Agotado'),
          ])
            ButtonSegment(
              value: status,
              icon: Icon(availabilityIcon(status), size: 18),
              label: Text(label),
            ),
        ],
        selected: {_status},
        onSelectionChanged: (values) =>
            setState(() => _status = values.first),
      ),
    ),
  ],
),
const SizedBox(height: 18),
TextFormField(
  controller: _comment,
  maxLength: 80,
  decoration: const InputDecoration(
    labelText: 'Comentario (opcional)',
    hintText: 'Ej. Ya no hay lomo, solo ají de gallina',
    prefixIcon: Icon(Icons.chat_bubble_outline_rounded),
  ),
),
const SizedBox(height: 8),
FilledButton.icon(
  onPressed: _submit,
  icon: const Icon(Icons.send_rounded),
  label: const Text('Publicar aviso'),
),
```

```
              ],
            ),
          ),
        ),
      ),
    );
  }
}

class _Label extends StatelessWidget {
  const _Label({required this.icon, required this.text});

  final IconData icon;
  final String text;

  @override
  Widget build(BuildContext context) => Row(
    children: [
      Icon(icon, size: 20, color: AppColors.orange),
      const SizedBox(width: 8),
      Text(text, style: AppText.display(18)),
    ],
  );
}
```

Anexo Q. test/widget_test.dart

```
import 'package:flutter_test/flutter_test.dart';

import 'package:menu_radar/main.dart';

void main() {
  testWidgets('La app arranca y muestra la barra de navegación', (tester) async {
    await tester.pumpWidget(const MenuRadarApp());

    expect(find.text('Radar'), findsWidgets);
    expect(find.text('Comunidad'), findsWidgets);
  });
}
```

Referencias

Flutter. (s. f.). Layouts in Flutter. https://docs.flutter.dev/ui/layout

Flutter. (s. f.). Layout widgets. https://docs.flutter.dev/ui/widgets/layout

Flutter. (s. f.). Understanding constraints. https://docs.flutter.dev/ui/layout/constraints

Flutter. (s. f.). Material components. https://docs.flutter.dev/ui/widgets/material

Flutter. (s. f.). CustomPaint class. https://api.flutter.dev/flutter/widgets/CustomPaint-class.html

Flutter. (s. f.). Hero animations. https://docs.flutter.dev/ui/animations/hero-animations

Flutter. (s. f.). Use a custom font. https://docs.flutter.dev/cookbook/design/fonts

Flutter. (s. f.). Build a form with validation. https://docs.flutter.dev/cookbook/forms/validation

Google. (s. f.). Material Design 3. https://m3.material.io

Google Fonts. (s. f.). Lilita One. https://fonts.google.com/specimen/Lilita+One

Google Fonts. (s. f.). Poppins. https://fonts.google.com/specimen/Poppins

Gordillo Mendoza, J. A., y Escobedo Ocaña, J. L. (s. f.). MenúRadar: aplicativo móvil universitario para la gestión en tiempo real de menús estudiantiles [Trabajo del curso, PE Lab 03, segunda unidad]. Universidad Nacional de San Agustín de Arequipa.